

Результат ревью:

Я отмечаю адекватный уровень твоей программистской подготовки. Ты используешь продвинутые концепции языка Python, библиотеки pandas и одной из основных библиотек машинного обучения scikit-learn. Это показывает твой уровень владения инструментами анализа данных и machine learning.

С комментированием никаких проблем нет. Твоих объяснений достаточно для понимания твоими коллегами хода мыслей. Ссылки из брифа проекта в нужных местах также помогают следовать за развитием событий и дают полную информацию касаясь того, что будет происходить в том или ином разделе проекта.

Мне было просто проверять твой проект, потому что он чётко структурирован, с использованием оглавления.

Зелёным цветом я отметил и прокомментировал удачные и элегантные решения, на которые ты можешь опираться в будущих проектах.

Жёлтым цветом я выделил то, что в следующий раз можно сделать по-другому. Ты можешь учесть эти комментарии при выполнении будущих заданий или доработать проект сейчас.

Красных комментариев, которые бы не позволяли мне принять твой проект в его нынешнем состоянии - нет.

ПРОЕКТ ПРИНЯТ!

Подводя итог: отличная работа, молодец! Желаю тебе успехов в учёбе, желаю, чтобы проекты сдавались или сразу, или после первой итерации! Удачи!

Если же после факта принятия проекта у тебя возникнут дополнительные вопросы – я никуда не пропал. Мой адрес эл/почты: anepomnjashiy@gmail.com (<mailto:anepomnjashiy@gmail.com>), имя в Пачке: anepomnjashiy, имя в Телеграмме: https://t.me/aleks_nep (https://t.me/aleks_nep).

p.s. я тебе на твой адрес эл/почты сейчас вышлю некоторые материалы из своей электронной библиотеки - надеюсь они будут тебе полезными в учёбе.

Ввод []:

Ввод []:

Приветствую Артемий!

Меня зовут Александр Непомнящий и я буду проверять твой проект. Предлагаю общаться на «ты», но, если это не удобно - мы вполне можем перейти на «вы».

Вкратце обо мне: я работаю в сфере информационных технологий, аналитика (анализ) данных - это и мой рабочий функционал и личная, интересная мне область для саморазвития. Используемый стек: python, pandas, графические библиотеки matplotlib/seaborn/plotly. Сфера применения: маркетинговая аналитика в медицинской клинике. Если вдруг есть вопрос – что делает аналитик в коммерческой медицине – то вот короткий ответ. Самый главный вопрос, на который мы хотим получить ответ (и который в дальнейшем приведёт к перераспределению рекламного бюджета, плюс напрямую повлияет на разработку и предложению рынку новых лечебных программ) - это КТО ОН, НАШ КЛИЕНТ? Фактически это задача (на нашем внутреннем языке формулируется как построение и анализ «ПОРТРЕТА КЛИЕНТА»). Визуализируем и пытаемся придать смысл половозрастным данным наших клиентов, кто из какого региона и в какие времена года к нам приезжает. Учитываем количество приездов. Учитываем на какую программу этот конкретный клиент приехал в первый раз и как в дальнейшем меняются его предпочтения. Пытаемся построить модель оттока и спрогнозировать, когда клиент уйдёт в отток. Смотрим цифры (суммы трат клиентов на отдельных группах медицинских программ). Анализируем медицинские услуги, которые входят в путёвку, а также что, в каких количествах и на какую сумму клиент покупает дополнительно. Очень вкратце как-то так.

Теперь к сути: основная цель моей проверки - пройти по шагам, по логике твоего решения и обсудить те моменты, которые можно изменить, улучшить, довести до идеала. Такой формат работы двух программистов (аналитиков) ты или уже встречал(а), или встретишь на своей новой работе. Кто-то такой процесс называет "код-ревью", ну а как по мне, так это обмен опытом двух коллег, когда каждый из нас становится лучше в процессе дискуссии.

Свои мысли, советы, комментарии и вопросы я буду оставлять в текстовых ячейках (markdown) ПОД твоим кодом. Для твоего удобства все эти заметки будут "цветными", как на примерах ниже:

"Зелеными" комментариями я буду отмечать твой элегантный ("красивый") код и/или удачные решения, на которые ты можешь смело опираться в будущих проектах.

Отлично! 👍

Отличная находка, элегантный код ...

"Жёлтые" комментарии - это мои размышления, предложения, что-то, что на следующей итерации можно сделать по-другому. Твоя реакция на "жёлтые" комментарий необходима и подразумевает или правки, или реакцию или аргументированное согласие/не согласие. Я искренне верю, что наш двусторонний диалог под "желтым" комментарием - это то, от чего твой проект только выиграет.

Комментарий 📝

Здесь я бы предложил использовать Мои аргументы в пользу этого ...

"Красные" комментарий будут появляться, когда что-то пойдёт не так: например, код будет выдавать ошибку. Или, когда в проекте не будет ответов на основные вопросы, ради которых весь анализ и выполнялся. "Красные" комментарии необходимо будет исправить, прежде чем я смогу принять проект.

Необходимо исправить ✖

В этой ячейке код работает не так как задумано ...

Пожалуйста, не перемещай, не изменяй и не удаляй мои комментарии. Это поможет мне оперативно выполнить повторную проверку твоего проекта.

Давай работать над проектом в диалоге: если ты что-то меняешь в проекте по моим рекомендациям — пиши об этом, лучше всего в новой ячейке, сразу под моим комментарием. Если для своих комментариев ты будешь использовать какой-то заметный цвет, то мне будет легче отследить твои изменения, например так:

КОММЕНТАРИЙ СТУДЕНТА

Текст твоего комментария ...

Распространенные типичные недочёты в проектах студентов:

Неработоспособный код. Проверь проект на работоспособность перед отправкой на проверку. Это важный навык, который тебе пригодится как на собеседовании, при выполнении тестового задания, так и перед совещанием у руководства компании.

Проект не структурирован. В проекте должна быть структура, т.е. названия разделов и подразделов должны быть записаны в ячейках Markdown.

Отсутствие описания, целей проекта, его задач, выводов. В описании должны быть озвучены поставленные задачи, содержание датасета, промежуточные и финальные выводы. Как правило, заказчик читает только финальный вывод, поэтому важно добавить информацию о проведенном исследовании, применить форматирование текста, в общем, сделать красиво, чтобы было понятно, что проведено настоящее исследование и выводу можно верить.

"Длинный код". Код и комментарии, которые прячутся за правым концом ячейки нужно переносить на новую строку. Твои будущие коллеги не оценят необходимость постоянного скроллинга. Согласно PEP8: <https://www.python.org/dev/peps/pep-0008/> длина строки не должна превышать 79 символов.

Твои коллеги ценят и твое и своё время, поэтому "подсвечивай" свои комментарии в ячейках Markdown. Подробнее про синтаксис Markdown, с примерами, можно почитать здесь: https://paulradzkov.com/2014/markdown_cheatsheet/

Александр привет! Большое спасибо за развернутое ревью. Приступаю к корректировке.

КОММЕНТАРИЙ V2

Артемий - приветствую! Приступаю ко 2ой итерации проверки и если всё исправлено, то сразу приму твой проект!

Ввод []:

Ввод []:

Проект: Линейные модели в машинном обучении

Цель проекта

- Разработать модель машинного обучения, которая поможет управлять рисками и принимать объективное решение о покупке.
 - Первая будет прогнозировать возможный удой коровы (целевой признак Удой);
 - Вторая — рассчитывать вероятность получить вкусное молоко от коровы (целевой признак Вкус молока).

План проекта

1. Загрузка и изучение данных

- Загрузить данные из файлов:
 - /datasets/ferma_main.csv
 - /datasets/ferma_dad.csv
 - /datasets/cow_buy.csv
- Вывести первые строки и информацию, дополнительно проверка стандартизации оформления данных и наименование столбцов.

2. Предобработка данных

- Проверка данных на наличие пропусков и дубликатов. Проверка корректности типов данных у каждого признака.

3. Исследовательский анализ данных

- Проведение статистического анализа всех признаков.
- Построение графиков для каждого признака.
- Вывод о данных:
 - Наличие выбросов, необычное распределение количественных признаков и другие особенности.
 - Идеи по улучшению данных.

4. Проведение корреляционного анализа признаков в датасете

- Изучение взаимосвязи между признаками: расчет коэффициентов корреляции между всеми признаками.
- Построение диаграммы рассеяния scatterplot для признака "Удой", кг и всех количественных признаков с учетом значения категориальных признаков.
- Выводы о взаимосвязи между признаками:
 - Какой тип взаимосвязи между целевым и входными признаками?
 - Есть ли нелинейные связи между целевым и входными признаками? Нужно ли вводить в модель дополнительные признаки, чтобы это учесть?
 - Наблюдается ли мультиколлинеарность между входными признаками?

5. Задача регрессии

- Подготовить данные датасета `ferma_main.csv` и обучить на них первую модель `LinearRegression`.
 - Использовать в качестве целевого признак Удой, кг.
 - Отобрать признаки, которые подходят для модели линейной регрессии, аргументировать выбор. Опирайтесь не только на наличие или отсутствие линейной связи между признаками, но и на причинно-следственные связи между урожаем коров и группами признаков из описания датасета.
 - Разбить данные на тренировочную и тестовую выборки.
 - Кодировать категориальные признаки с помощью `OneHotEncoder`.
 - Масштабировать количественные признаки с помощью `StandardScaler`.
 - Обучить модель линейной регрессии на подготовленных данных.
 - Оценить качество модели на тестовой выборке: рассчитайте R^2 , проанализировать остатки с помощью графиков.
 - Оформить выводы, указать:
 - значение R^2 модели;
 - какие выводы следуют из анализа остатков;
 - нужны ли дополнительные исследования.
- Подготовить данные с учётом связи входных признаков с целевым и обучить на них вторую модель `LinearRegression`.
 - Оценить диаграммы рассеяния: возможно, в них можно увидеть, что некоторые входные признаки связаны с целевым нелинейно. Исключить нелинейность, чтобы эти признаки были информативны для линейной регрессии. Это можно сделать:
 - При обнаружении, что СПО и Удой связаны нелинейно. На их диаграмме рассеяния наблюдения сгруппированы в два кластера, причём есть такое значение СПО, которое можно считать границей между кластерами. Чтобы устранить нелинейность, преобразовать СПО в категориальный бинарный признак, используя эту границу.

- При обнаружении, что ЭКЕ и Удой также связаны нелинейно. Нелинейность между ними можно исключить, если ввести новый признак ЭКЕ в квадрате.
- Повторить шаги из пункта 1 и создать вторую модель: подготовить данные, обучить модель, оценить её качество и написать выводы.
- Добавить в данные новый признак и обучить на них третью модель LinearRegression.
 - Добавить признак "Имя Папы" из файла `ferma_dad.csv`, соединив его с остальными данными `ferma_main.csv` по ключу.
 - Устранить нелинейность между признаками.
 - Повторить шаги из пункта 1 и создать третью модель: подготовить данные, обучить модель, оценить её качество и написать выводы.
- Сравнить качество трёх моделей линейной регрессии, сопоставить их R^2 и остатки.
- Оценить качество каждой модели по трём метрикам: MSE, MAE, RMSE. Использовать функции для расчёта метрик.
 - Выбрать лучшую модель;
 - Указать доверительный интервал прогноза лучшей модели. Для этого рассчитать квантили ошибки модели на тренировочных данных.
- С помощью лучшей модели спрогнозировать удой коров, которых фермер хочет купить. Данные о них находятся в датасете `cow_buy.csv`.
 - Добавить недостающие признаки ЭКЕ (Энергетическая кормовая единица), Сырой протеин, г и СПО (Сахаро-протеиновое соотношение): взять из датасета `ferma_main.csv` средние значения каждого из признаков и увеличить их на 5%.
 - Выполнить подготовку данных, аналогичную той, которую делали для тренировочной выборки.
 - Получить прогноз удоя коров.

6. Задача классификации

- Подготовить данные датасета `ferma_main.csv` и обучить на них модель LogisticRegression.
 - Использовать в качестве целевого признак Вкус молока.
 - При необходимости преобразовать имеющиеся или добавить новые признаки.
 - Разбить данные на тренировочную и тестовую выборки.
 - Кодировать категориальные признаки с помощью OneHotEncoder.
 - Масштабировать количественные признаки с помощью StandardScaler.
 - Обучить модель логистической регрессии на подготовленных данных.
 - Оценить качество модели на тестовой выборке: рассчитать метрики `accuracy`, `recall`, `precision`.
 - Построить матрицу ошибок.
- Проанализировать модель логистической регрессии.
 - Объяснить, какая ошибка критичнее всего при покупке коров: первого или второго рода. Помнить, что главное для заказчика — вкусное молоко. Сформулировать ответ с учётом метрик `recall` и `precision`, выбрать более важную из них.
 - Свести критичную ошибку к нулю: для этого изменить порог принадлежности к классам. Ещё раз рассчитать значения метрик `recall` и `precision`. Какое значение порога мы выбрали? Как изменились значения метрик?
 - Оформить выводы: определить, что нужно сделать дальше, исходя из анализа модели?
- С помощью обученной модели спрогнозировать вкус молока коров, которых фермер хочет купить. Данные о них находятся в датасете `cow_buy.csv`.
 - Выполнить подготовку данных, аналогичную той, которую делали для тренировочной выборки.
 - Получить прогноз вкуса молока коров.

7. Сделать итоговые выводы

- Добавить прогнозы вкуса молока и удою коров «ЭкоФермы» в датафрейм с коровами, покупку которых рассматривает фермер.
- Отбрать коров, для которых спрогнозирован удой более 6000 килограммов в год и вкусное молоко.
- Коротко описать результаты работы моделей для фермера.
 - Сколько коров он может купить из стада на продажу с минимальным риском для себя? Почему?
 - Какие ещё рекомендации можно ему дать?
- Сделать выводы о моделях линейной регрессии и логистической регрессии.
 - Какие способы улучшения качества регрессионной модели вы использовали в этой задаче?
 - Возможно ли ещё улучшить результаты регрессионной модели? Если можно, то как? Если нет, то почему?

Описание данных

1. Файл `ferma_main.csv` содержит данные о стаде фермера на текущий момент. Описание данных:

- `id` — уникальный идентификатор коровы.
- Удой, кг — масса молока, которую корова даёт в год (в килограммах).
- ЭКЕ (Энергетическая кормовая единица) — показатель питательности корма коровы.
- Сырой протеин, г — содержание сырого протеина в корме (в граммах).
- СПО (Сахаро-протеиновое соотношение) — отношение сахара к протеину в корме коровы.
- Порода — порода коровы.
- Тип пастбища — ландшафт лугов, на которых паслась корова.
- Порода папы_быка — порода папы коровы.
- Жирность, % — содержание жиров в молоке (в процентах).
- Белок, % — содержание белков в молоке (в процентах).
- Вкус молока — оценка вкуса по личным критериям фермера, бинарный признак (вкусно, не вкусно).
- Возраст — возраст коровы, бинарный признак (менее2лет, более2лет).
 - Признаки в датасете `ferma_main.csv` можно разделить на группы.
- Характеристики коровы: `id`, Порода, порода папы_быка, Возраст.
- Характеристики корма: ЭКЕ (Энергетическая кормовая единица), Сырой протеин, г, СПО (Сахаро-протеиновое соотношение).
- Характеристика пастбища: Тип пастбища.
- Характеристики молока: Удой, кг, Жирность, %, Белок, %, Вкус молока.
 - Такое разделение поможет вам понять, как взаимосвязаны признаки в датасете.

2. Файл `ferma_dad.csv` хранит имя папы каждой коровы в стаде фермера. Описание данных:

- `id` — уникальный идентификатор коровы.
- Имя Папы — имя папы коровы.

3. Файл `cow_buy.csv` — это данные о коровах «ЭкоФермы», которых фермер хочет изучить перед покупкой. Описание данных:

- Порода — порода коровы.
- Тип пастбища — ландшафт лугов, на которых паслась корова.
- Порода папы_быка — порода папы коровы.

- Имя_папы — имя папы коровы.
- Текущая_жирность,% — содержание жиров в молоке (в процентах).
- Текущий_уровень_белок,% — содержание белков в молоке (в процентах).
- Возраст — возраст коровы, бинарный признак (менее2лет, более2лет).
 - Данные в файлах ferma_main.csv и cow_buy.csv похожи, но у датасета cow_buy.csv несколько особенностей.
- Содержание белков и жиров в молоке указано на момент продажи — когда «ЭкоФерма» давала коровам свой корм.
- Параметры корма ЭКЕ (Энергетическая кормовая единица), Сырой протеин, г и СПО (Сахаро-протеиновое соотношение) отсутствуют. Технологи заказчика пересмотрели подход к кормлению: для новых коров планируется увеличить значения каждого из этих параметров на 5%.
- Кроме того, отсутствуют признаки Удой, кг и Вкус молока. Это целевые признаки, именно их предстоит предсказать.

ОТЛИЧНО! 👍

Ты делаешь хорошую работу уже в начале нашего проекта по Data Science. Твой подход к описанию всего проекта уже с самого начала – это плюс. Ты ясно и четко формулируешь цель и план нашего исследования, что дает нам основу для выполнения дальнейших этапов (... слона надо есть по кусочкам ... - такой мудрый совет есть на предмет того как подступиться к решению большой и сложной на первый взгляд задаче).

Твое описание того, как ты будешь двигаться дальше, также впечатляет. Твои действия по обработке и структурированию данных в начальных этапах – это правильный шаг. Также отмечу, что твое умение систематизировать данные с самого начала проекта позволяет нам лучше понимать характер данных и уже сейчас делать некоторые предварительные наброски. Такой подход в начале аналитического/Дата Сайенс проекта говорит о твоей компетентности и ставит нас на "правильную дорогу" для успешного исследования.

1. Загрузка данных

Загружаем необходимые библиотеки.


```
Ввод [1]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import math
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler
from scipy.sparse import hstack
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, recall_score, precision_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import precision_recall_curve
from sklearn.preprocessing import LabelEncoder
```

ОТЛИЧНО! 👍

Отлично, все импорты собраны в начале "тетрадки"!

В Python есть своя философия. Именно отсюда берутся определенные правила и стандарты. В пайтон такое соглашение называли PEP8.

Соответствие PEP8 позволяет видеть основную структуру вашего кода другим разработчикам. Не нужно привыкать к другому оформлению кода. Если вы привыкли как-то оформлять код, а увидели что-то совершенно другое, то вы поймете.

Официальная документация: <https://peps.python.org/pep-0008/> (<https://peps.python.org/pep-0008/>)

Основные моменты и рекомендации:

- Используйте пробелы, разрешается использование табов, но смешивание запрещено. Отступ соответствует 4 пробелам. Хорошие редакторы кода автоматически заменяют табы на пробелы.
- Вокруг операторов +, -, =, , /, * должны быть пробелы.
- Строка не должна превышать 79 символов
- Ну и наконец про импорты: импорты можно делать только по одному на строку! При этом из одного модуля можно доставать несколько нужных объектов. Порядок тоже важен:
 - импорты из стандартной библиотеки
 - импорты из сторонних библиотек
 - импорты пользовательских модулей текущего проекта (например если выносишь функции в отдельный модуль)

И нежелательно их перемешивать с константами и переменными.

p.s. состав стандартных библиотек Python здесь: [The Python Standard Library \(https://docs.python.org/3/library/\)](https://docs.python.org/3/library/)

Ввод [2]: `pip install -q phik`

Note: you may need to restart the kernel to use updated packages.

Комментарий 🙋

Полезный ключ **-q** (он "подавит" служебные сообщения об установке, которых может быть достаточно много - вплоть до нескольких экранов прокрутки).

Добавил

Ввод [3]: `import phik`

Далее загружаем и изучаем датафреймы.

Ввод [4]: `ferma_main = pd.read_csv('/datasets/ferma_main.csv', sep = ';')
ferma_dad = pd.read_csv('/datasets/ferma_dad.csv', sep = ';')
cow_buy = pd.read_csv('/datasets/cow_buy.csv', sep = ';')`

ОТЛИЧНО! 👍

Отлично. Импортировали требуемые для анализа библиотеки и загрузили данные. Сейчас посмотрим на их состояние и если всё ОК, сможем приступить к построению моделей....

p.s. при установке дополнительных библиотек (через ! pip install ...) можно использовать полезный ключ **-q** (он "подавит" служебные сообщения об установке, которых может быть достаточно много - вплоть до нескольких экранов прокрутки)

p.p.s. При считывании данных из файла здорово перестраховывать себя от ошибок, связанных, например, с неверным указанием пути к файлу. А иногда бывает, что работаешь с файлом локально, выгружаешь его на сервер, ожидая, что он будет принимать данные, которые лежат на том же сервере, а код падает с ошибкой, потому что путь к файлу не поменялся с локального на серверный.

Для этого, например, можно использовать конструкцию **try-except** : сначала пробуешь локальный путь, при возникновении ошибки используется серверный путь (подробнее можешь почитать тут: <https://pythonworld.ru/typy-dannyx-v-python/isklyucheniya-v-python-konstrukciya-try-except-dlya-obrabotki-isklyuchenij.html> (<https://pythonworld.ru/typy-dannyx-v-python/isklyucheniya-v-python-konstrukciya-try-except-dlya-obrabotki-isklyuchenij.html>)).

Но еще лучше использовать библиотеку `os` - её использование позволит тебе проверять существование указанных директорий (что может быть актуально при одновременной работе на локальном и сетевом окружении) и загружать данные из существующей директории, избегая ошибок. Как пример:

```
import os

pth1 = '/folder_1/data.csv'
pth2 = '/folder_2/data.csv'

if os.path.exists(pth1):
    query_1 = pd.read_csv(pth1)
elif os.path.exists(pth2):
    query_1 = pd.read_csv(pth2)
else:
    print('Something is wrong')
```

Ещё на этапе считывания данных можно спарсить дату: за это действие отвечает параметр `parse_dates` метода `read_csv()`, в него нужно передать список с названием полей-дат, и в большинстве случаев дата будет корректно преобразована в нужный формат сразу.) Также на этапе считывания данных задать индекс-столбец- за это действие отвечает параметр `index_col`.

Ввод [5]:

display(ferma_main.head(3))
display(ferma_dad.head(3))
display(cow_buy.head(3))

	id	Удой, кг	ЭКЕ (Энергетическая кормовая единица)	Сырой протеин, г	СПО (Сахаро- протеиновое соотношение)	Порода	Тип пастбища	порода папы_быка	Жирность,%	Белок,%	Вкус молока	Возраст
0	1	5863	14,2	1743	0,89	Вис Бик Айдиал	Равнинное	Айдиал	3,58	3,076	вкусно	более_2_лет
1	2	5529	12,8	2138	0,89	Вис Бик Айдиал	Равнинные	Соверин	3,54	3,079	вкусно	менее_2_лет
2	3	5810	14	1854	0,885	РефлешнСоверинг	Холмистое	Соверин	3,59	3,074	не вкусно	более_2_лет

	id	Имя Папы
0	1	Буйный
1	2	Соловчик
2	3	Барин

	Порода	Тип пастбища	порода папы_быка	Имя_папы	Текущая_жирность,%	Текущий_уровень_белок,%	Возраст
0	Вис Бик Айдиал	холмистое	Айдиал	Геркулес	3,58	3,076	более_2_лет
1	Вис Бик Айдиал	равнинное	Соверин	Буйный	3,54	3,081	менее_2_лет
2	РефлешнСоверинг	равнинное	Соверин	Барин	3,59	3,074	более_2_лет

Переименуем столбцы во всех датафреймах, для удобства работы с ними.

```
Ввод [6]: ferma_main = ferma_main.rename(columns={
    'Удой, кг': 'Milk',
    'ЭКЕ (Энергетическая кормовая единица)': 'Energy',
    'Сырой протеин, г': 'Protein',
    'СПО (Сахаро-протеиновое соотношение)': 'SP0',
    'Порода': 'Breed',
    'Тип пастбища': 'Pasture',
    'порода папы_быка': 'Father breed',
    'Жирность,%': 'Fat',
    'Белок,%': 'Prot in milk',
    'Вкус молока': 'Flavor',
    'Возраст': 'Age'
})

ferma_dad = ferma_dad.rename(columns={'Имя Папы': 'Father name'})

cow_buy = cow_buy.rename(columns={
    'Порода': 'Breed',
    'Тип пастбища': 'Pasture',
    'порода папы_быка': 'Father breed',
    'Имя папы': 'Father name',
    'Текущая жирность,%': 'Fat',
    'Текущий уровень белка,%': 'Prot in milk',
    'Возраст': 'Age'
})

cow_buy = cow_buy.replace({'холмистое': 'Холмистое', 'равнинное': 'Равнинное'})
```

Комментарий 🖱️

Небольшой коммент к этому коду: если у нас в датафрейме будет несколько десятков столбцов, то процесс переименовывания может занять много времени и череват ошибками в названиях. Рекомендация: присмотреться к изменению названий с помощью регулярных выражений (например как на скрине ниже).

Вводная статья на тему использования регулярных выражений - тут: <https://habr.com/ru/articles/349860/> (<https://habr.com/ru/articles/349860/>)

```
In [9]: #приведение названий колонок датафреймов в snake_case
dfs = [
    dt_market_file,
    dt_market_monou
```

Комментарий 👉

Советую названия столбцов в датафреймах привести к "змеинному регистру"!

p.s и приведение названий столбцов к нижнему регистру и изменение типов данных столбцов - это отдельные шаги в общем пайплайне исследовательского анализа данных (EDA). Ниже ссылки, описывающие взгляд на процесс EDA "сверху":

1. <https://waksoft.susu.ru/2021/07/23/issledovatel'skij-analiz-dannyh-v-python-rukovodstvo-dlya-novichkov-na-2021-god/> (<https://waksoft.susu.ru/2021/07/23/issledovatel'skij-analiz-dannyh-v-python-rukovodstvo-dlya-novichkov-na-2021-god/>).
2. <https://www.kaggle.com/code/ekami66/detailed-exploratory-data-analysis-with-python> (<https://www.kaggle.com/code/ekami66/detailed-exploratory-data-analysis-with-python>).
3. <https://www.datacareer.de/blog/exploratory-data-analysis-in-python/> (<https://www.datacareer.de/blog/exploratory-data-analysis-in-python/>).

Полностью с тобой согласен. Не подумал, что столбцов может быть 1000:) Если можно, то сейчас оставлю как есть.

КОММЕНТАРИЙ V2

Не подумал, что столбцов может быть 1000:) - у меня в текущем рабочем их около 70. И записей около 6 млн (на горизонте 10 лет, с 01.0.1 2016).

Если можно, то сейчас оставлю как есть - да, здесь, в учебном это ОК. Мы же учимся.

```
Ввод [7]: # проверка
print('Столбцы ferma_main:', ferma_main.columns)
print('--' * 50)
print('Столбцы ferma_dad:', ferma_dad.columns)
print('--' * 50)
print('Столбцы cow_buy:', cow_buy.columns)
```

```
Столбцы ferma_main: Index(['id', 'Milk', 'Energy', 'Protein', 'SP0', 'Breed', 'Pasture',
                           'Father breed', 'Fat', 'Prot in milk', 'Flavor', 'Age'],
                           dtype='object')
```

```
-----
Столбцы ferma_dad: Index(['id', 'Father name'], dtype='object')
```

```
-----
Столбцы cow_buy: Index(['Breed', 'Pasture', 'Father breed', 'Father name', 'Fat',
                        'Prot in milk', 'Age'],
                        dtype='object')
```

Вывод:

Данные не плохие. Столбцы преименовали для удобства. Так же изменили регистр первой буквы в столбце тип пастибща(Pasture).

2. Предобработка данных

Проверим данные на наличие пропусков и дубликатов. А так же проверим тип данных.

```
Ввод [8]: print('Пропуски в ferma_main:', ferma_main.isna().sum())
print('-' * 50)
print('Пропуски в ferma_dad:', ferma_dad.isna().sum())
print('-' * 50)
print('Пропуски в cow_buy:', cow_buy.isna().sum())
```

```
Пропуски в ferma_main: id          0
Milk          0
Energy        0
Protein        0
SP0           0
Breed         0
Pasture        0
Father breed   0
Fat           0
Prot in milk   0
Flavor         0
Age           0
dtype: int64
-----
Пропуски в ferma_dad: id          0
Father name    0
dtype: int64
-----
Пропуски в cow_buy: Breed         0
Pasture        0
Father breed    0
Father name     0
Fat            0
Prot in milk    0
Age            0
dtype: int64
```

Пропуски необнаружены.

Комментарий 🙋

В наших данных пропусков НЕТ, НО ЕСЛИ БЫ БЫЛИ я стараюсь дать подробный коммент-видение на тему обработки пропусков данных (так как понимаю, что несмотря на кажущуюся простоту - не очень просто понять чем ("заглушкой"? / медианой? / средним? / по комбинации столбцов?) заполнять? а может удалять?

По результатам проверки 5+ тысяч проектов за 3 года у меня родилось небольшое эссе на тему "удаления vs. спасения" данных, как раз на основе тех ситуаций и вопросов, которые мне коллеги задавали, которые мы обсуждали, уточняли и исправляли ...

► [Подробнее ...](#)

Да, в целом согласен с тобой. В предыдущих проектах, где в изначальных данных были пропуски, мог потратить 1-3 часа на поиск решения с ними. Тк простой вариант удалить или "заглушить" мало-перспективный.

```
Ввод [9]: print('Дубликаты ferma_main:', ferma_main.duplicated().sum())
print('-' * 50)
print('Дубликаты ferma_dad:', ferma_dad.duplicated().sum())
```

Дубликаты ferma_main: 5

Дубликаты ferma_dad: 0

```
Ввод [10]: ferma_main = ferma_main.drop_duplicates().reset_index(drop = True)
```

Необходимо исправить ❌

Датафрейм "с коровами для предсказаний" - это ведь по сути тестовая выборка, данные для предсказаний (для наших моделей). А в тестовой выборке мы не правомочны что либо удалять - эти данные поступают на вход обученной модели и по ним должны быть сделаны "предсказания".

Нет никакой разницы есть ли дубли в тестовой выборке или нет: **это данные по которым ДОЛЖНЫ БЫТЬ сделаны "предсказания"**.

Один раз мне студент задал такой вопрос:

КОММЕНТАРИЙ СТУДЕНТА

То есть, свершилось чудо, и у нас 4 коровы с абсолютно одинаковыми параметрами?

Убрал корректировку дубликатов в cow_buy

КОММЕНТАРИЙ V2

Исправлено.

```
Ввод [11]: # проверка
print('Дубликаты ferma_main:', ferma_main.duplicated().sum())
```

Дубликаты ferma_main: 0

Комментарий 🙋

Хочу отметить, что в общем случае даже если **df.duplicated().sum() == 0** - это может совсем не означать, что в данных нет дубликатов.

В реальных рабочих датасетах скорее всего не будет полных дубликатов, но вот неявные наверняка будут встречаться, соответственно с ними надо будет уметь работать. Для поиска неявных дублей обычно используют параметр **subset** у функции `duplicated()`. Правда здесь, в нашем проекте, надо поразмыслить над тем какие комбинации столбцов могут говорить о реальных дублях в данных.

Дубликаты удалены. Сделаем проверку неявных дубликатов.

Ввод [12]: `print(ferma_main.nunique())`

```
id          629
Milk        528
Energy      55
Protein     401
SP0         12
Breed        2
Pasture      3
Father breed 3
Fat         50
Prot in milk 15
Flavor       2
Age          2
dtype: int64
```

Видим что, значений в столбце Pasture - 3, а должно быть 2 и Father breed аналогично.

Ввод [13]: `print(ferma_main['Pasture'].unique())`
`print(ferma_main['Father breed'].unique())`

```
['Равнинное' 'Равнинные' 'Холмистое']
['Айдиал' 'Соверин' 'Айдиалл']
```

Ввод [14]: `ferma_main['Pasture'] = ferma_main['Pasture'].replace({'Равнинные': 'Равнинное'})`
`ferma_main['Father breed'] = ferma_main['Father breed'].replace({'Айдиалл': 'Айдиал'})`

ОТЛИЧНО! 👍

Отлично, молодец, обнаружено 2 "разночтения" (породы + типы пастбищ) по факту одних и тех же категорий в признаках.

Ввод [15]: `print(ferma_main.nunique())`

```
id          629
Milk        528
Energy      55
Protein     401
SP0         12
Breed        2
Pasture      2
Father breed 2
Fat         50
Prot in milk 15
Flavor       2
Age          2
dtype: int64
```

Переходим к просмотру типов данных.

Ввод [16]: `display('Типы данных ferma_main:', ferma_main.dtypes)`
`print('-' * 50)`
`display('Типы данных ferma_dad:', ferma_dad.dtypes)`
`print('-' * 50)`
`display('Типы данных cow_buy:', cow_buy.dtypes)`

'Типы данных ferma_main:'

```
id          int64
Milk        int64
Energy      object
Protein     int64
SP0         object
Breed       object
Pasture     object
Father breed object
Fat         object
Prot in milk object
Flavor      object
Age         object
dtype: object
```

'Типы данных ferma_dad:'

id int64

```
Ввод [17]: ferma_main['Energy'] = ferma_main['Energy'].str.replace(',', '.').astype(float)
ferma_main['SPO'] = ferma_main['SPO'].str.replace(',', '.').astype(float)
ferma_main['Fat'] = ferma_main['Fat'].str.replace(',', '.').astype(float)
ferma_main['Prot in milk'] = ferma_main['Prot in milk'].str.replace(',', '.').astype(float)
# ferma_main['Flavor'] = ferma_main['Flavor'].map({'вкусно': 1, 'не вкусно': 0})
ferma_main['Age'] = ferma_main['Age'].map({'более_2_лет': 1, 'менее_2_лет': 0})

cow_buy['Fat'] = cow_buy['Fat'].str.replace(',', '.').astype(float)
cow_buy['Prot in milk'] = cow_buy['Prot in milk'].str.replace(',', '.').astype(float)
cow_buy['Age'] = cow_buy['Age'].map({'более_2_лет': 1, 'менее_2_лет': 0})

le = LabelEncoder()
ferma_main['Flavor'] = le.fit_transform(ferma_main['Flavor'])
ferma_main['Flavor'] = 1 - ferma_main['Flavor']
```

```
Ввод [18]: print(ferma_main['Flavor'].head())
print(ferma_main['Age'].head())
```

```
0    1
1    1
2    0
3    0
4    1
Name: Flavor, dtype: int64
0    1
1    0
2    1
3    1
4    0
Name: Age, dtype: int64
```

ОТЛИЧНО! 👍

Отлично, привели типы ... явно числовые типы, но при этом имеющие тип "строка" приведены к числовому типу.

Кроме того, что теперь признаки в нашем датафрейме имеют корректные типы данных - это действие ещё и вот по какой причине полезно: оптимизация объёма больших датасетов может помочь значительно сократить объём памяти (занимаемый датасетом). Это особенно важно, если у вас ограничены вычислительные ресурсы или если требуется работать с огромными объёмами данных.

Функция `pd.to_numeric()` с параметром `downcast='integer'` позволяет снизить тип числовых данных до наименьшего возможного размера без потери информации. Например, преобразование целочисленных значений из типа `int64` до `int32`, `int16` или `int8` может существенно сократить использование памяти, особенно если значения в датасете могут быть представлены более короткими типами данных.

Однако при использовании этой оптимизации следует быть осторожным. Преобразование типов данных может привести к потере точности или диапазона значений, особенно если числа в датасете имеют большой разброс или могут выходить за пределы нового типа данных. Поэтому перед использованием функции `pd.to_numeric()` с параметром `downcast='integer'`, рекомендуется проверить, что сокращение размера типа не приведет к потере важной информации для конкретной задачи.

Комментарий 🖱️

```
Ввод [17]: ferma_main['Energy'] = ferma_main['Energy'].str.replace(',', '.').astype(float)
ferma_main['SPO'] = ferma_main['SPO'].str.replace(',', '.').astype(float)
ferma_main['Fat'] = ferma_main['Fat'].str.replace(',', '.').astype(float)
ferma_main['Prot in milk'] = ferma_main['Prot in milk'].str.replace(',', '.').astype(float)
ferma_main['Flavor'] = ferma_main['Flavor'].map({'вкусно': 1, 'не вкусно': 0})
ferma_main['Age'] = ferma_main['Age'].map({'более_2_лет': 1, 'менее_2_лет': 0})

cow_buy['Fat'] = cow_buy['Fat'].str.replace(',', '.').astype(float)
cow_buy['Prot in milk'] = cow_buy['Prot in milk'].str.replace(',', '.').astype(float)
cow_buy['Age'] = cow_buy['Age'].map({'более_2_лет': 1, 'менее_2_лет': 0})
```

Кодирование таргета нельзя делать "руками":

- при появлении новых значений категориальных признаков, не предусмотренных твоим решением, модель или упадёт, или будет генерировать ошибки;
- такое решение не сделаешь частью пайплайна, а все современные решения базируются на них.

Поэтому кодирование нужно делать только с применением обучаемых трансформеров. Для кодирования целевого признака нужно использовать `LabelEncoder` (сокращаю до `LE`): <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html> (<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html>).

p.s. если стоит вопрос в том, что тебе не нравится, что `LE`, например, присваивает классу "снизилась" метку 0, а "прежний уровень" - метку 1, то значения меток можно инвертировать с помощью небольшого лайфхака:

```
import numpy as np

# Пример преобразованных классов
target = np.array([0, 1, 1, 0, 1, 0])
```

Принял, спасибо за подсказку. Переделал.

```
Ввод [19]: # проверка
display('Типы данных ferma_main:', ferma_main.dtypes)
print('-' * 50)
display('Типы данных ferma_dad:', ferma_dad.dtypes)
print('-' * 50)
display('Типы данных cow_buy:', cow_buy.dtypes)
```

```
'Типы данных ferma_main:'
```

id	int64
Milk	int64
Energy	float64
Protein	int64
SPO	float64
Breed	object
Pasture	object
Father breed	object
Fat	float64
Prot in milk	float64
Flavor	int64
Age	int64
dtype:	object

```
-----
'Типы данных ferma_dad:'
```

```
id          int64
```

Вывод по шагу 2:

- Пропуски обнаружены не были.
- Дубликаты явные и неявные устранены.
- Типы данных отредактированы.

ОТЛИЧНО! 👍

В этом разделе всё ОК: необходимые после загрузки данных действия (1. загрузка данных, 2. требования PEP8, 3. дубликаты, 4. анализ уникальных значений в признаках 5. пропуски) выполнены.

Этот вывод результат твоей работы описывает и подытоживает. Здесь всё супер, молодец!

3. Исследовательский анализ данных

Делаем статистический анализ.

```
Ввод [20]: display(ferma_main.describe())
display(ferma_main.describe(exclude=['number']))
```

	id	Milk	Energy	Protein	SPO	Fat	Prot in milk	Flavor	Age
count	629.00000	629.000000	629.000000	629.000000	629.000000	629.000000	629.000000	629.000000	629.000000
mean	315.00000	6188.750397	14.543879	1923.364070	0.913116	3.603657	3.075671	0.589825	0.825119
std	181.72094	1644.795489	1.306408	182.956251	0.032203	0.168656	0.002549	0.492257	0.380167
min	1.00000	5204.000000	10.900000	1660.000000	0.840000	2.990000	3.069000	0.000000	0.000000
25%	158.00000	5751.000000	13.500000	1771.000000	0.890000	3.590000	3.074000	0.000000	1.000000
50%	315.00000	6133.000000	14.700000	1888.000000	0.930000	3.650000	3.076000	1.000000	1.000000
75%	472.00000	6501.000000	15.600000	2062.000000	0.940000	3.720000	3.077000	1.000000	1.000000
max	629.00000	45616.000000	16.800000	2349.000000	0.960000	3.750000	3.085000	1.000000	1.000000

	Breed	Pasture	Father breed
count	629	629	629
unique	2	2	2
top	Вис Бик Айдиал	Холмистое	Айдиал
freq	373	351	318

Комментарий 📌

describe() играет важную роль в понимании распределения статистик признаков в данных. Отсюда можно увидеть аномалии, выбросы. Просто "посмотреть" на эти таблички недостаточно, обязательно нужен вывод по анализу результата describe().

describe() в pandas - это мощный метод, который предоставляет статистическое описание данных. Вот несколько причин, почему describe() важен:

1. Общее понимание данных: describe() предоставляет общий обзор статистических характеристик, таких как среднее значение, стандартное отклонение, минимум, максимум и квартили. Это позволяет быстро понять основные характеристики данных.
2. Выявление выбросов: Анализ статистических показателей может помочь выявить аномалии или выбросы в данных. Например, если среднее значение сильно отличается от медианы, это может указывать на наличие выбросов.
3. Понимание распределения данных: describe() также предоставляет информацию о распределении данных через квартили, что может быть полезным для понимания формы распределения.
4. Быстрый анализ пропущенных значений: Метод также выводит количество непустых значений для каждого столбца, что может помочь в определении пропущенных данных.

```
Ввод [21]: # заменили аномальное значение, на среднее
ferma_main['Milk'] = ferma_main['Milk'].replace(45616, 6188)
```

ОТЛИЧНО! 👍

Есть, отлично, выброс по удою (45 тонн молока в год - это фантастика!) обнаружен и обработан в таблице с текущим стадом фермера.

Комментарий 👉

```
Ввод [20]: # заменили аномальное значение, на среднее
ferma_main['Milk'] = ferma_main['Milk'].replace(45616, 6188)
```

Артемий - починаешь почему "хардкод" это плохо?

Да конечно. Была мысль назначить новую переменную и через нее уже заменить значения, но с другой стороны тогда больше строк и памяти. Если можно, скажи как это сделать самым правильным способом.

КОММЕНТАРИЙ V2

но с другой стороны тогда больше строк и памяти. - ну количество строк увеличившееся - это неважно. Какая разница сколько их. Памяти в компьютерах нынешних столько. что это тоже не проблема.

Если можно, скажи как это сделать самым правильным способом. - кодом Питона:

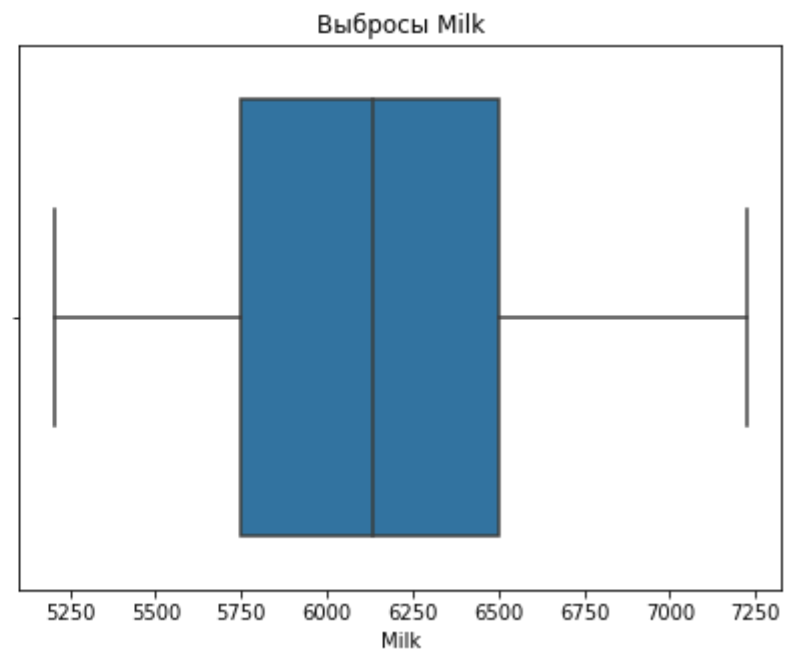
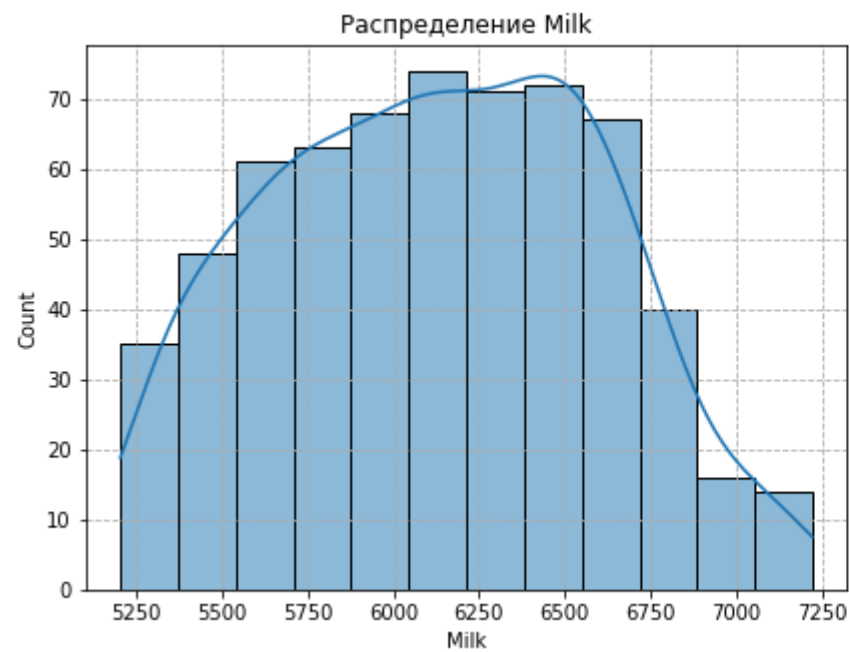
1. найти самое большое число в столбце (`max()`).
2. подумать как его обрабатывать. Если как ты решаешь (отрезать 1ю цифру), то перевести число в строку (`str()`), отрезать 1ый символ от строки (используя `slice`), перевести строку обратно в число (`int()`).

```
Ввод [22]: num_cols = [col for col in ferma_main.select_dtypes(include='number').columns
                if col != 'id']
cat_cols = ferma_main.select_dtypes(include = 'object').columns.tolist()

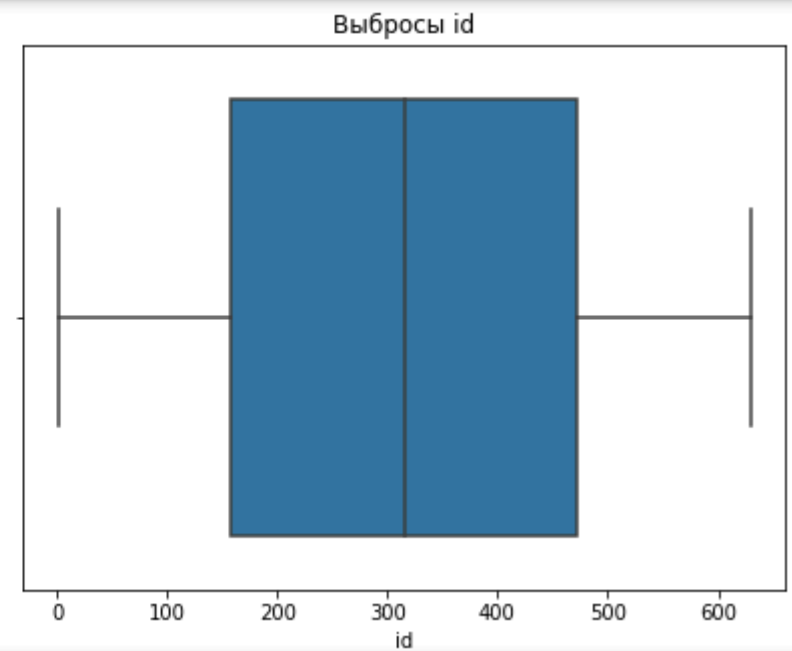
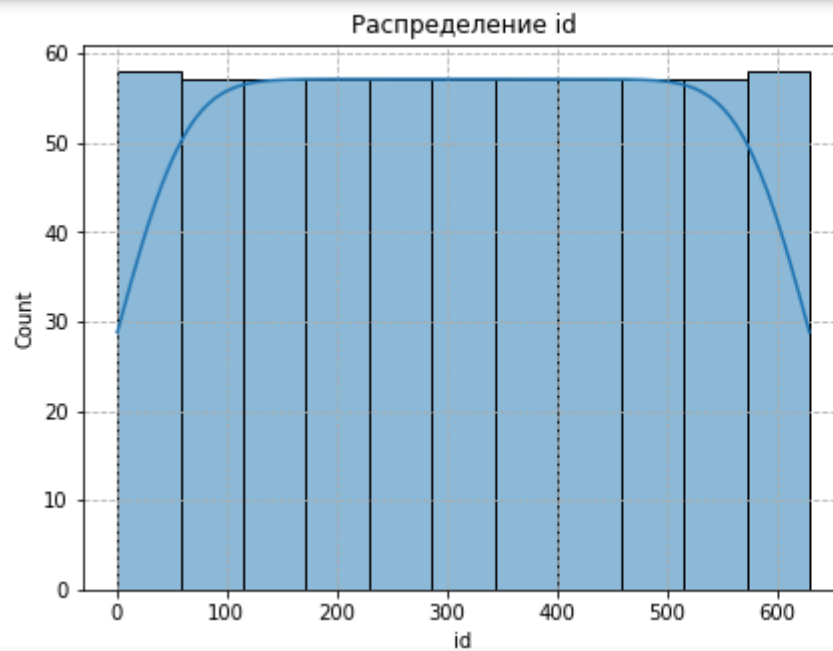
for col in num_cols:
    plt.figure(figsize=(15, 5))
    plt.subplot(1, 2, 1)
    sns.histplot(ferma_main[col], kde=True)
    plt.title(f'Распределение {col}')
    plt.grid(True, linestyle = '--')
    plt.subplot(1, 2, 2)
    sns.boxplot(x=ferma_main[col])
    plt.title(f'Выбросы {col}')
    plt.show()

    Q1 = ferma_main[col].quantile(0.25)
    Q3 = ferma_main[col].quantile(0.75)
    IQR = Q3 - Q1
    outliers = ferma_main[(ferma_main[col] < (Q1 - 1.75*IQR)) | (ferma_main[col] > (Q3 + 1.75*IQR))]
    print(f'{col}: {len(outliers)} выбросов')

for col in cat_cols:
    plt.figure(figsize=(10, 4))
    sns.countplot(x=col, data=ferma_main)
    plt.title(f'Распределение {col}')
    plt.grid(True, linestyle = '--')
    plt.xticks(rotation=45)
    plt.show()
    print(ferma_main[col].value_counts());
```



Необходимо исправить ❌



Ну и о чем нам говорит этот график? Он полезен нам в EDA?

id из графического анализа нужно удалить (это просто инкремент, порядковые номера коров).

id убран

КОММЕНТАРИЙ V2

Исправлено.

ОТЛИЧНО! 👍

Просто супер: график подписан, оси подписаны. Не часто на самом деле такой скрупулёзный подход к оформлению встречается. Молодец! Я закончил физико-математический факультет Петрозаводского государственного университета. Нас там с первого курса приучили - если на сдачу лабораторной работы по физике приходишь с графиком, на котором не подписано общее название, названия осей (с единицами измерений), единичная шкала - всё сдача заканчивается не начавшись. Или если у моей лабораторной работы отсутствует вывод: что делали? по какой методике? что "намеряли"? как интерпретировали? какой финальный вывод? - всё, тоже на выход. Ну и в принципе сейчас, на текущей работе: код, который написан в тетрадке мало кому понятен и не сильно кому интересен. Важнее детальные выводы и тренды, которые попадут на слайд презентации в PowerPoint. Именно поэтому столь важное значение в аналитике данных придается промежуточным выводам и уж тем более финальному.

(МЕТОДИЧЕСКИЕ МАТЕРИАЛЫ ПО ОФОРМЛЕНИЮ ГРАФИКОВ):

При работе с графиками важно соблюдать следующие правила:

1. Стоит подписать названия осей X и Y.
2. Также стоит дать название графику.
3. Технические комментарии типа `<AxesSubplot: xlabel='datetime'>` стоит убирать - это можно сделать, поставив точку с запятой в последней строке кода в блоке визуализации, либо сделать последней строкой кода этого блока метод `plt.show()`.
4. Графики нужно сделать больше - графики дефолтного размера не очень удобны для чтения. Удобнее будет настроить параметр `plt.rcParams["figure.figsize"] = (x,y)` сразу в ячейке импорта библиотек, и все графики проекта будут одного размера.

Несколько таких нехитрых приёмов повысят качество твоих визуализаций в проекте. **И здесь следует сказать что ты об правилах оформления всё знаешь! МОЛОДЕЦ!**

Комментарий 👉

Хотя на графиках подписи есть (об этом похвальный комментарий выше), но я всё-таки хочу обратить твое внимание, что одинаковые "обезличенные" подписи на всех графиках - это не ОК. Графики принято подписывать так, чтобы подписи **отражали суть** изображенного на осях, а также подписи должны содержать и единицы измерения того, что на осях изображено.

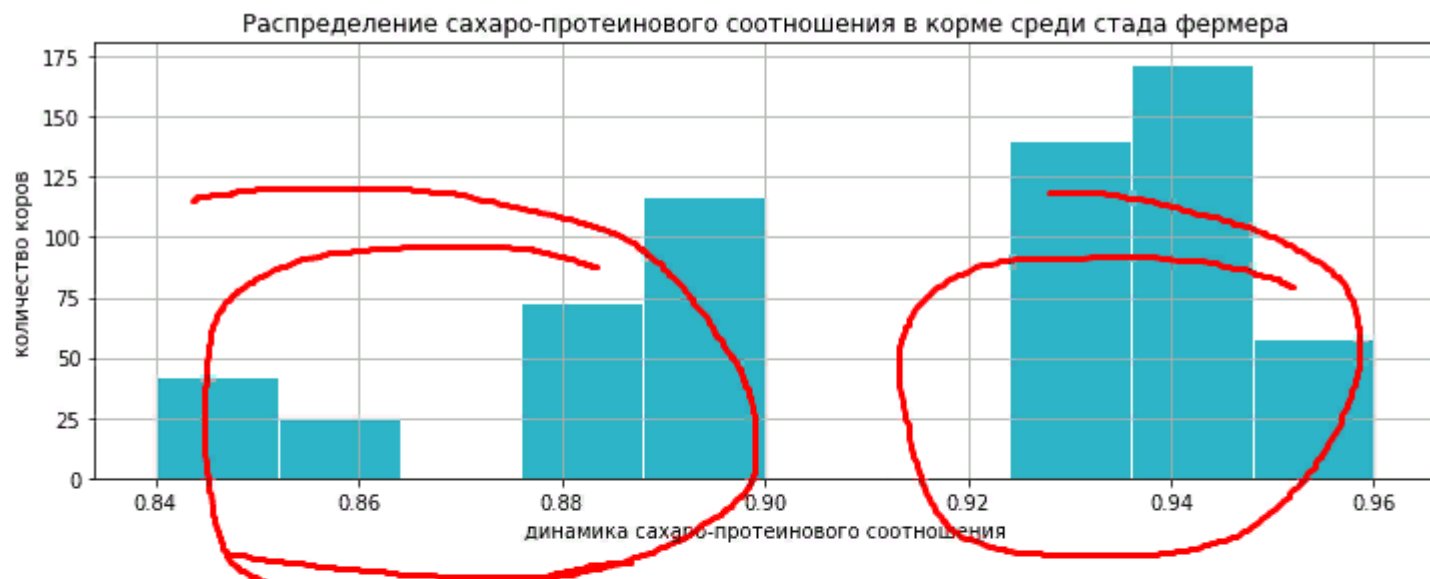
Если графики строиться в цикле, то выход такой: вручную создать словарь: "ключ" - название признака по которому на соответствующем шаге будет строиться график, "значения" - список названий ([название графика, название оси ОХ, название оси ОУ]). Далее пробегаем в цикле по названиям признаков (в данных). Сопоставляем название признака с ключом словаря, а названия подписей (графика и 2х осей) берем из соответствующего этому ключу значения.

ОТЛИЧНО! 👍

Отлично! Боксплот рядом с гистограммой - это отлично! Гистограмма будет показывать нам распределение количественного признака, а боксплот даст информацию о **возможных выбросах** (пишу **возможных**, потому что нужно анализировать бизнесовый смысл значений показанных как выбросы ... просто так удалять то, что показывает боксплот за пределами 1,5 IQR от границ "ящика" - неверно) и основных статистиках (квартилях) признака.

Комментарий 👉

(на примере моего скрина ниже) Вот здесь уже видно, что этот признак - кандидат на перевод в категориальную шкалу (из 2х или из 3х элементов - что надо проверять качеством модели итоговой).



Согласен с тобой полностью на счет возможности перевода SPO в категориальную шкалу, но есть ли сейчас в этом смысл, если по ТЗ на второй модели мы переведем SPO в бинарный вид.

Комментарий 🙋

(на примере моего скрина ниже) Признак "жирность" тоже кажется потенциально той фичей, которую имело бы смысл перевести в категориальную шкалу: "низкая"-"средняя"-"высокая" (хотя в отличии от СПО это в ТЗ не требуется ... но попытаться сделать можно. После этого посмотреть как это скажется на качестве модели). На примере моего(!) скрина: возможные отсечки по категориям ...

```
Ввод [23]: low = ferma_main['Fat'].quantile(0.33)
medium = ferma_main['Fat'].quantile(0.66)

print(f'''
Границы категорий:
- Низкая: < {low:.3f}%
- Средняя: {low:.3f}% – {medium:.3f}%
- Высокая: > {medium:.3f}%
''')
```

```
Границы категорий:
- Низкая: < 3.620%
- Средняя: 3.620% – 3.710%
- Высокая: > 3.710%
```

```
Ввод [24]: def categorize_fat(value):
            if value < low:
                return 'low'
            elif value < medium:
                return 'medium'
            else:
                return 'high'

ferma_main['Fat'] = ferma_main['Fat'].apply(categorize_fat)
```

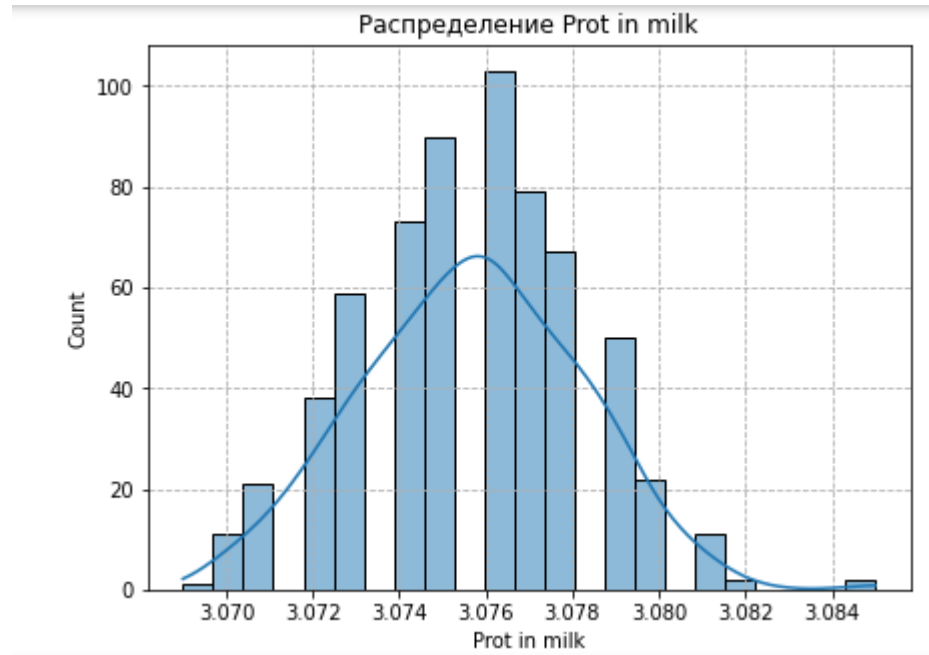
```
Ввод [25]: ferma_main.head()
```

```
Out[25]:
```

	id	Milk	Energy	Protein	SPO	Breed	Pasture	Father breed	Fat	Prot in milk	Flavor	Age
0	1	5863	14.2	1743	0.890	Вис Бик Айдиал	Равнинное	Айдиал	low	3.076	1	1
1	2	5529	12.8	2138	0.890	Вис Бик Айдиал	Равнинное	Соверин	low	3.079	1	0
2	3	5810	14.0	1854	0.885	РефлешнСоверинг	Холмистое	Соверин	low	3.074	0	1
3	4	5895	12.4	2012	0.885	РефлешнСоверинг	Холмистое	Айдиал	low	3.075	0	1
4	5	5302	12.8	1675	0.885	Вис Бик Айдиал	Равнинное	Соверин	high	3.073	1	0

Перевел столбец Fat в категориальные значения.

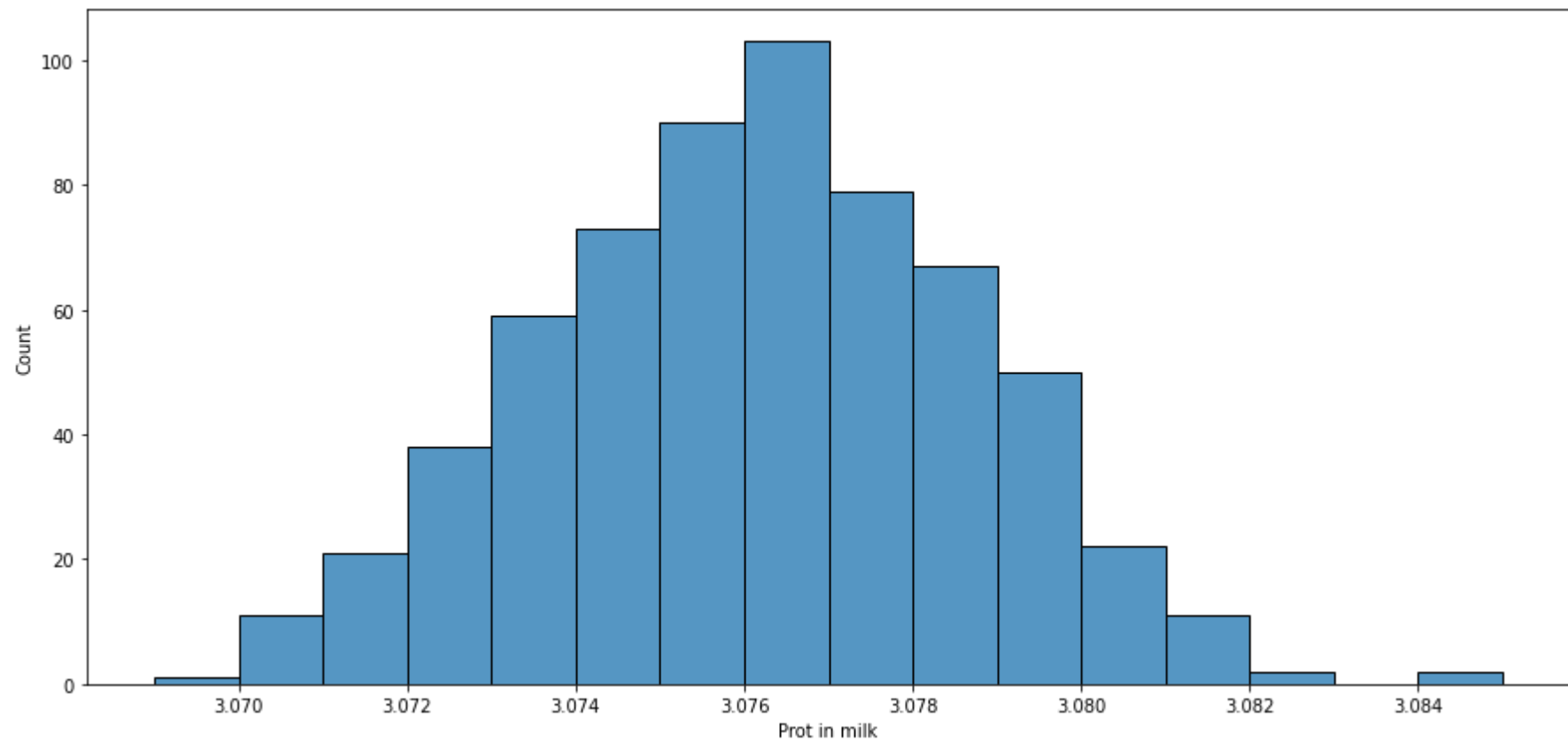
Необходимо исправить ✖



"Провалы" в значениях гистограммы реально существуют? Или они появляются ввиду ошибочно выбранного значения bins?

Есть ряд техник, которые позволяют "плюс/минус" определиться с числом бинов, например **правило Стёрджеса**: число bins равно $\log_2(n) + 1$, где n - количество наблюдений.

```
Ввод [26]: # проверка 'Prot in milk'
plt.figure(figsize = (15, 7))
sns.histplot(
    ferma_main['Prot in milk'],
    bins = 16
);
```



```
Ввод [27]: ferma_main['Prot in milk'].value_counts()
```

```
Out[27]: 3.076    103
          3.075     90
          3.077     79
          3.074     73
          3.078     67
          3.073     59
          3.079     50
          3.072     38
          3.080     22
          3.071     21
          3.070     11
          3.081     11
          3.082      2
          3.085      2
          3.069      1
Name: Prot in milk, dtype: int64
```

Проблема на графике 'Prot in milk', была в неверном(неинформативном) значении бинов. Судя по значениям value_counts, пробелов в значениях у нас нет.

КОММЕНТАРИЙ V2

Отлично! Исправлено.

Распределение Milk - выбросов нет, средние значения от 6000 до 6500.

Распределение Energy - выбросов нет, преобладают более высокие значения.

Распределение Protein - выбросов нет, преобладают низкие значения. Распределение SPO - разделяются на 3 кластера, 0.84 - 0.86, 0.87 - 0.91, 0.93 - 0.96, выбросов нет.

Распределение Fat - неравномерное, выбросов нет, большинство коров имеют высокое содержание жирности в молоке.

Распределение Prot in milk - равномерное, с пробелами, выбросов нет.

Flavor - значения "1" (вкусно) преобладают.

Age - значения "1" (более 2 лет) преобладают значительно.

Breed - преобладает порода "Вис Бик Айдиал".

Pasture - преобладает "Холмистое".

Father breed - преобладает Айдиал, но распределение практически равномерное.

Комментарий 🖱️

Распределение Milk - нормальное, выбросов нет, средние значения от 6000 до 6500
Распределение Energy - нормальное, выбросов нет, преобладают более высокие значения
Распределение Protein - нормальное, выбросов нет, преобладают низкие значения.
0.87 - 0.91, 0.93 - 0.96, выбросов нет.
Распределение Fat - неравномерное, выбросов нет, большинство коров имеют высокие значения

В наших данных нет ни одного признака с нормальным распределением. Ты можешь проверить мои слова.

У нормального распределения не может быть никаких **скосов / смещений / мультимод / ассиметрии / выбросов / пиков** - нормальное распределение **АБСОЛЮТНО СИММЕТРИЧНОЕ**.

(Не)нормальность распределения данных проверяешь "на глаз", но это несколько некорректно - есть вероятность ошибки (понятно что не в нашем случае - тут всё ясно ... но вот в случае распределения похожего на Гауссиану). Что надо сделать чтобы строго подтвердить или опровергнуть нормальность распределения случайной величины?

Вот полезные материалы материалы на эту тему: <https://education.yandex.ru/handbook/data-analysis/article/bazovye-statisticheskie-testy> (<https://education.yandex.ru/handbook/data-analysis/article/bazovye-statisticheskie-testy>) и <https://education.yandex.ru/handbook/data-analysis/article/bazovye-statisticheskie-testy-v-python> (<https://education.yandex.ru/handbook/data-analysis/article/bazovye-statisticheskie-testy-v-python>)

Подтверждать нормальность (распределения) данных (на гистограмме) "на глаз" нельзя. Это неверно. Пожалуйста сделай это "строго". Вот полезные материалы материалы на эту тему: <https://education.yandex.ru/handbook/data-analysis/article/bazovye-statisticheskie-testy> (<https://education.yandex.ru/handbook/data-analysis/article/bazovye-statisticheskie-testy>) и <https://education.yandex.ru/handbook/data-analysis/article/bazovye-statisticheskie-testy-v-python> (<https://education.yandex.ru/handbook/data-analysis/article/bazovye-statisticheskie-testy-v-python>)

Комментарий 🖱️

Выводы по гистограммам и/или боксплотам (то что вижу я ... для сопоставления наших позиций):

- явная бимодальность в данных. Т.е. надо углублять в анализы подгрупп, прежде чем делать какие-то выводы.

- есть супервыброс по удою. Остальные точки на "ящиках с усами" - можно считать приемлимыми значениями на данном этапе. При моделировании (когда будем выполнять анализ остатков) посмотрим детальнее.
- признаки СПО и Жирность: есть подозрение, что их лучше перевести в категориальную шкалу, т.к. есть большая ассиметрия - скорее всего тут прячутся две-три группы.

ОТЛИЧНО! 👍

Данные загружены, отлично.

Хорошо, что применяешь метод `.head()/.sample()` таким образом в память не будет загружен весь датасет, объем которого может быть огромным.

Методы `.info()`, `.describe()`, `.value_counts()`, `.unique()` как правило часто встречаются и обязательны к применению в задачах по исследованию данных, ведь с их помощью можно получить первые инсайты.

Итак, мы провели предварительный анализ данных, проверили данные на наличие пропусков и затем обработали их (аргументированно заполнили / удалили / оставили как есть). Провели графический анализ. Сделали и озвучили выводы. Теперь можно приступать собственно к моделированию и ответам на вопросы проекта.

ОТЛИЧНО! 👍

Этот шаг спасет твою аналитику (и карьеру): классический кейс - к аналитику (неважно: DA-аналитику, DS-аналитику, ML-инженеру) прибегает его начальник с горящими глазами: «Срочно нужно посчитать метрики, отчет нужен вчера, Я ЖДУUUUUUUUU!!!!!!!!!!!!!!!!!!!!!!». Аналитик лезет в сырые данные, быстро собирает дашборд ... и всё. Ни тебе проверки распределений, ни поиска выбросов, ни визуализации. Начальник на следующий день презентует эти цифры как истину в последней инстанции. А потом на них строятся бизнес-решения стоимостью десятки-сотни миллионов рублей ...

► **[ТАК ДЕЛАТЬ НЕЛЬЗЯ! НИКОГДА!!!](#)**

4. Корреляционный анализ

ОТЛИЧНО! 👍

Смотришь скоррелированность признаков - отлично! Об этом чуть ниже подробнее напишу....

Phi_K correlation представляет собой инструмент для вычисления и визуализации корреляции между категориальными и количественными переменными. Она может быть полезной в анализе данных перед построением моделей машинного обучения, так как позволяет обнаружить скрытые взаимосвязи между переменными, которые могли бы остаться незамеченными при обычных методах корреляционного анализа. Преимущества использования **phik**:

1. **Учет категориальных переменных:** Традиционные методы вычисления корреляции, такие как коэффициент корреляции Пирсона, применяются только к количественным переменным. **phik** позволяет вычислять корреляцию между категориальными и количественными переменными, что делает его мощным инструментом для анализа разнообразных типов данных.
2. **Обнаружение нелинейных зависимостей:** **phik** способен обнаруживать нелинейные связи между переменными, которые могли бы быть упущены традиционными методами. Это особенно важно в анализе данных, где зависимости могут быть сложными и нелинейными (что может влиять на твой выбор признаков для моделей МО).
3. **Подходит для больших наборов данных:** **phik** обладает хорошей масштабируемостью, что позволяет использовать его для анализа больших наборов данных без серьезного снижения производительности.
4. **Визуализация результатов:** Библиотека предоставляет инструменты для визуализации матрицы корреляции между переменными, что помогает лучше понимать зависимости в данных.

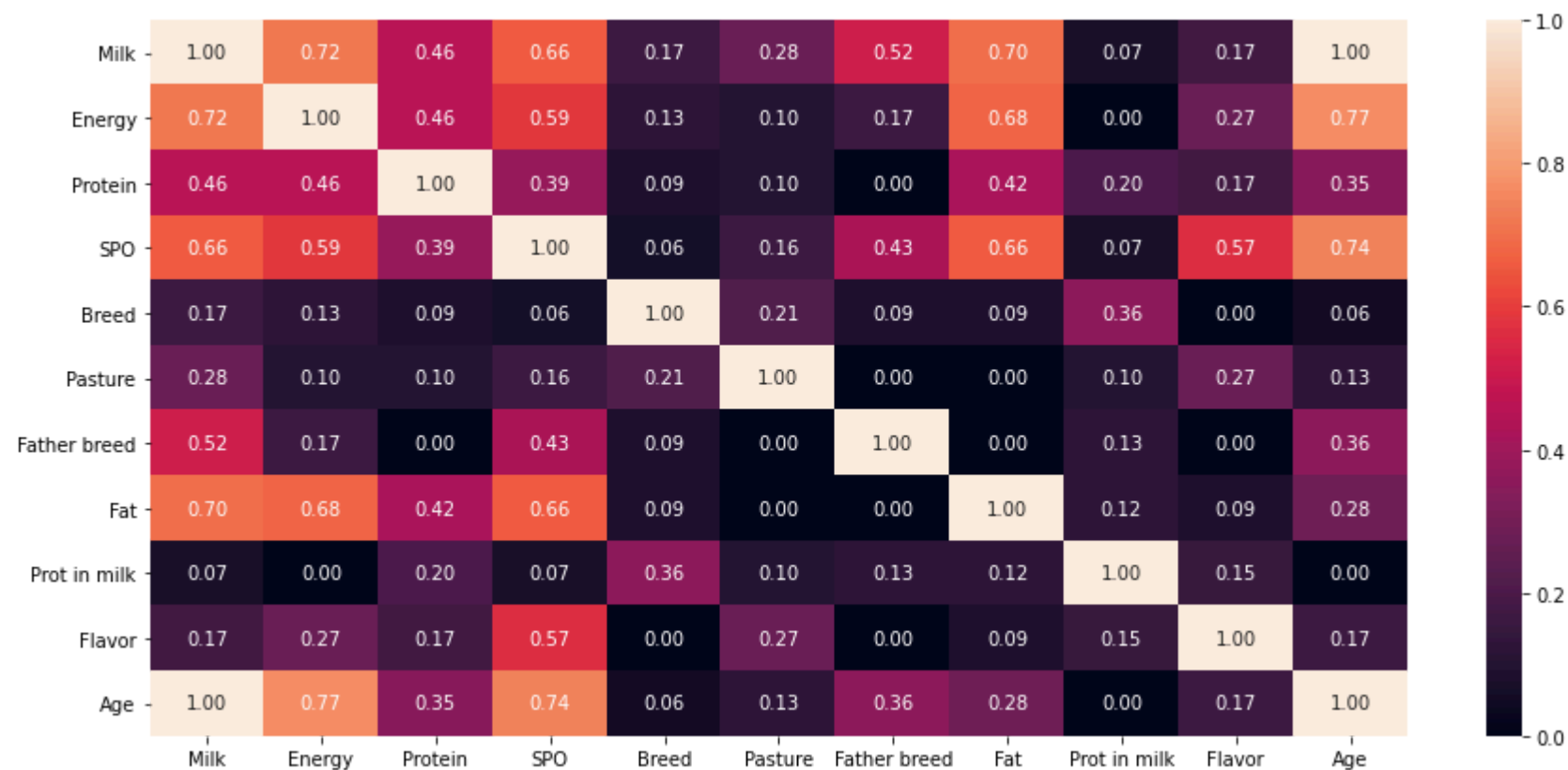
p.s. у этой библиотеки есть ограничение с которым я например сталкивался: когда в даных есть столбцы с очень большим количеством уникальных значений, то **phik** может "упасть" с сообщением об ошибке (суть которого в том, что не хватает памяти для обработки такого количества "уников"). При этом на компьютере у меня 32 Гб памяти и большая часть её не занята. Я подозреваю что это некое внутреннее ограничение библиотеки.

Приступаем к корреляционному анализу. Сделаем матрицу корреляций между всеми признаками.

```
Ввод [28]: num_cols = ferma_main.select_dtypes(include = 'number').columns.tolist()
cat_cols = ferma_main.select_dtypes(include = 'object').columns.tolist()
interval_cols = ['Milk', 'Energy', 'Protein', 'SPO', 'Prot in milk']

ferma_main_no_id = ferma_main.drop(columns = ['id'])
ferma_main_phik = ferma_main_no_id.phik_matrix(interval_cols=interval_cols)

plt.figure(figsize = (15, 7))
sns.heatmap(
    data = ferma_main_phik,
    annot = True,
    fmt = '0.2f'
);
```



Необходимо исправить ❌

```
Ввод [22]: num_cols = ferma_main.select_dtypes(include = 'number').columns.tolist()
cat_cols = ferma_main.select_dtypes(include = 'object').columns.tolist()

ferma_main_no_id = ferma_main.drop(columns = ['id'])
ferma_main_phik = ferma_main_no_id.phik_matrix()

plt.figure(figsize = (15, 7))
sns.heatmap(
    data = ferma_main_phik,
    annot = True,
    fmt = '0.2f'
);

interval columns not set, guessing: ['Milk', 'Energy', 'Protein', 'SPO', 'Fat', 'Prot in milk', 'Flavor', 'Age']
```

При использовании **phik** крайне важно настроить параметр **interval_cols**, передав ему список с названиями непрерывных признаков: если этого не сделать, библиотека своими силами будет пытаться разнести признаки по типам, и она часто делает в этом ошибки - от этого в свою очередь мы получаем некорректную оценку корреляций, так как от типа признака зависит метод оценки корреляции.

Возможный вопрос: ... я все равно не понимаю зачем в **interval_cols** передавать именно непрерывные признаки, после моих исправлений, матрица практически не изменилась ...

Ответ: итак, зачем в **interval_cols** передавать именно непрерывные признаки? Библиотека **phik** рассчитывает обобщённую корреляцию между переменными, в том числе категориальными и числовыми. Для числовых (интервальных) признаков **phik** автоматически бинит значения в интервалы (бины), чтобы можно было строить кросс-таблицы и считать корреляцию.

Непрерывные признаки (например, "выручка", "длительность" с большим количеством уникальных значений) хорошо подходят для биннинга, потому что их значения равномерно распределены по диапазону, и разделение на бины даёт осмысленное представление о распределении значений.

Дискретные признаки (например, "количество акционных покупок" с 10 уникальными значениями) при биннинге могут терять смысл: если уникальных значений мало, то бины будут крупными и не отразят реальную структуру данных.

Почему матрица почти не изменилась после исправлений? Если убирать из **interval_cols** признаки с малым числом уникальных значений (дискретные), а оставить только действительно непрерывные, то биннинг станет более корректным, но если в данных изначально мало действительно непрерывных признаков, то итоговая матрица **phik** действительно изменится не сильно. Это связано с тем, что **phik** хорошо работает и с категориальными переменными, и если большинство признаков дискретные, то их вклад в матрицу корреляций будет примерно одинаковым независимо от того, как они определены (категориальными или интервальными).

*Почему не стоит передавать дискретные признаки в **interval_cols**?*

- Потеря информации: Если признак с 20 уникальными значениями разбить на 10 бинов, то в каждом бине окажется по 2 значения — это грубое усреднение и потеря детализации.
- Смысл биннинга: Для дискретных признаков биннинг не добавляет смысла, а наоборот, может исказить структуру данных.

- Категориальные признаки: Лучше оставить такие признаки категориальными — `phik` корректно считает корреляции между категориальными переменными без биннинга.

Исправил.

КОММЕНТАРИЙ V2

Исправлено.

Наиболее сильная корреляция у Milk(Удой) - Age, Fat, Father Breed, SPO, Protein, Energy.

У Flavor(Вкус) - SPO и немного Energy и Pasture, но нужно проверять оба признака дополнительно.

Комментарий 📌

Выводы с учётом того, что уже выполнено тобой в проекте, выше (то что вижу я ... для сопоставления наших позиций):

- очень высоких корреляций (0.9 +) не видно, скорее всего мультиколлинеарности нет.
- есть подозрение на нелинейность связи удоя с "спо" и "жирностью".
- по спо: постоянная зависимость на разных уровнях. до 0.91 - одна зависимость с целевой. После 0.91 - иная. Просто "видим" две группы (некоторые "видят" 3). Надо ввести новую категориальную переменную: "спо менее 0.9"/"спо более 0.9".
- По жирности: история такая же. Наблюдается три зоны с разной зависимостью с целевой: до 3.5, и ещё две - на моем графике все 3 выделены красными прямоугольниками.

Строим график scatterplot.

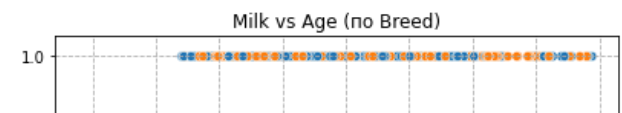
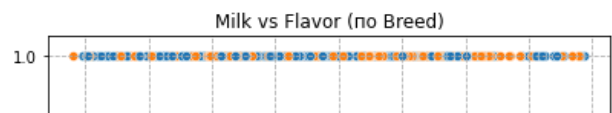
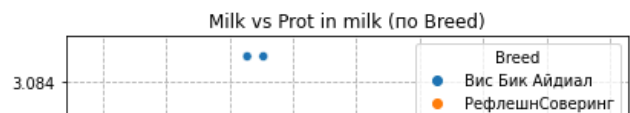
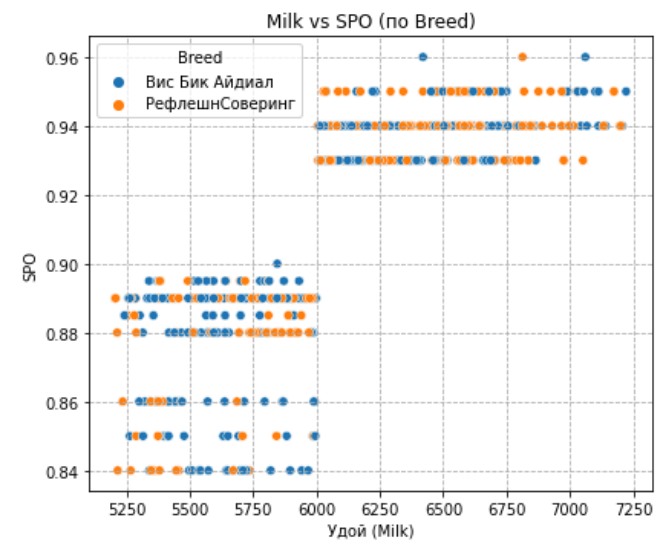
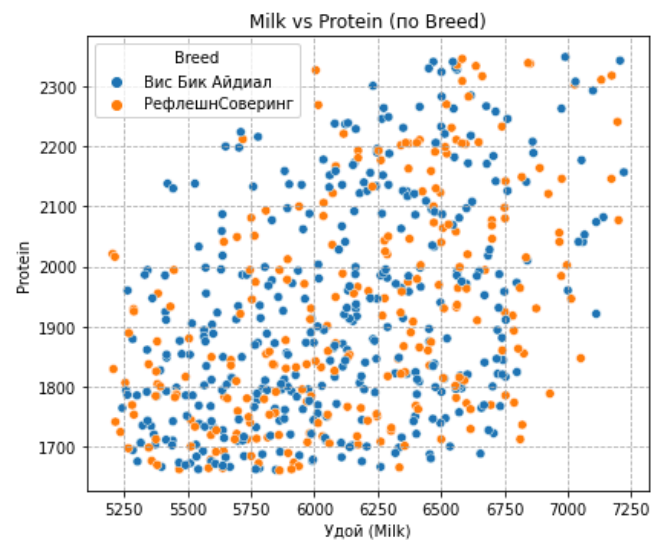
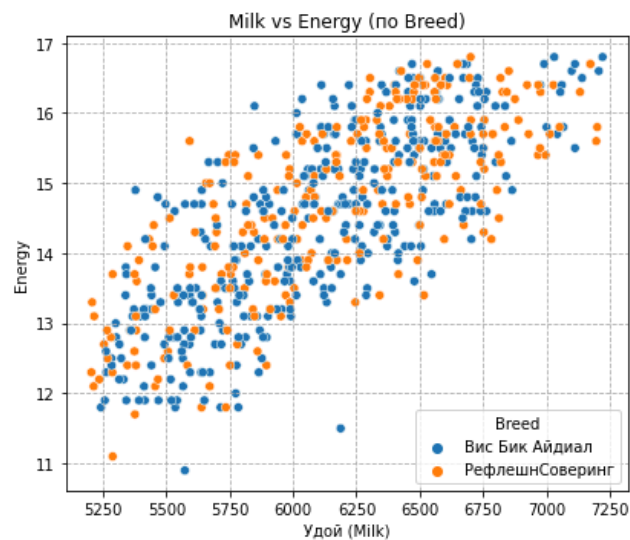
```

Ввод [29]: num_cols = ferma_main_no_id.select_dtypes(include = 'number').columns.tolist()
cat_cols = ferma_main_no_id.select_dtypes(include = 'object').columns.tolist()
num_cols = [col for col in num_cols if col != 'Milk']

for cat in cat_cols:
    n_plots = len(num_cols)
    n_rows = math.ceil(n_plots / 3)
    plt.figure(figsize=(18, 5 * n_rows))

    for i, num in enumerate(num_cols, 1):
        plt.subplot(n_rows, 3, i)
        sns.scatterplot(
            data = ferma_main_no_id,
            x = 'Milk',
            y = num,
            hue = cat)
        plt.title(f'Milk vs {num} (по {cat})')
        plt.xlabel('Удой (Milk)')
        plt.ylabel(num)
        plt.grid(True, linestyle = '--')
    plt.tight_layout()
    plt.show()

```



На примере моего(!) скрина отчётливо видно что ЭКЕ с удоем связано нелинейно (квадратично) и возведение ЭКЕ в степень улучшит наверняка линейную модель.

Также видно что СПО имеет смысл перевести в категорию (на уровне между 0.91 - 0.92, примерно ... тоже на примере моего скрина).

р.с. Эти 2 графика (с "добавлениями" на них) - они к тому, что волшебства в решении сделать СПО категориальным, а ЭКЕ квадратичным нет. Это решение мы принимаем не потому что нас осенило, а потому что нам графики это показывают. Это первый важный момент который не всегда мы чётко понимаем. Но есть и 2ой важный момент. Визуализации в ДА/ДС проекте - это не просто "красивые картинки". Это отображение данных в том виде, который позволяет РЕШЕНИЯ ПРИНИМАТЬ.



график похож на квадратичную функцию, параболу:

$$y = x ** 2$$

Промежуточный вывод:

Milk - Energy:

- линейная зависимость, нормальное распределение, порода папы быка "Айдиал" эффективнее реагирует на повышение Energy(энергетическая кормовая единица). Тип пастбища особо не влияет на удой, но более высокие значения достигаются на холмистом типе.

Milk - Protein:

- нормальное распределение, нет линейной связи, опять же у породы папы быка "Айдиал" более высокие значения удоя, на холмистом типе.

Milk - SPO:

- кластерное распределение, с четкой границей (от 0.84 до 0.9 удой до 6000 кг, от 0.93 до 0.96 удой выше 6 кг), аналогично порода папы быка "Айдиал" и тип пастбища холмистое.

Milk - Fat:

- нелинейное распределение, с отдельными кластерами. От 3.0 до 3.5 наименьшее распределение, 3.6 и отдельно 3.7 наибольшие значения по количеству. Порода папы быка и тип пастбища, аналогично.

Milk - Prot in milk:

- нормальное распределение, связи нет, у породы "Вис Бик Айдиал", в большинстве случаев, более высокое значение. Это же подтверждается по породе папы быка и типу пастбища, аналогично предыдущим наблюдениям.

Milk - Flavor:

- равномерное распределение, связи нет, вкусное молоко вместе бОльшим удоём, встречается чаще по породе папы быка "Айдиал" и на холмистом типе пастбища.

Milk - Age:

- линейная связь, распределение нормальное, больше половины значений имеют значение 1(старше 2 лет), остальное аналогично.

Вывод: Идеальная модель - порода "Вис Бик Айдиал", порода папы быка "Айдиал", тип пастбища "Холмистое". А так же на Удой влияет увеличение Energy(ЭКО) и возраст коровы более 2 лет.

Далее делаем проверку на мультиколлинеарность.

```
Ввод [30]: from statsmodels.stats.outliers_influence import variance_inflation_factor

numeric_data = ferma_main_no_id.select_dtypes(include=['number']).drop(columns=['Milk'])

vif_data = pd.DataFrame()
vif_data["feature"] = numeric_data.columns
vif_data["VIF"] = [variance_inflation_factor(numeric_data.values, i)
                   for i in range(numeric_data.shape[1])]

print("VIF анализ:")
display(vif_data.sort_values("VIF", ascending=False))
```

VIF анализ:

	feature	VIF
2	SPO	2049.260645
3	Prot in milk	1206.629975
0	Energy	304.959042
1	Protein	141.278512
5	Age	8.792444
4	Flavor	2.565965

Комментарий 🙋

```
print( VIF анализ: )  
display(vif_data.sort_values)
```

VIF анализ:

	feature	VIF
2	SPO	2157.403684
4	Prot in milk	1364.138637
3	Fat	884.359094
0	Energy	333.751615

На счет темы мультиколлинеарности, если честно не совсем понял. Но, по твоему выводу я понял, что ее нет, закоментю код на удаление лишних столбцов.

КОММЕНТАРИЙ V2

На счет темы мультиколлинеарности, если честно не совсем понял - в Инете статей масса на эту тему, пробуй читать, со временем всё уложиться в голове.

```
Ввод [31]: # удаляем лишние признаки  
ferma_clean = ferma_main_no_id.drop(columns = ['Protein', 'Flavor'])
```

```
Ввод [32]: # from statsmodels.stats.outliers_influence import variance_inflation_factor  
  
# numeric_data = ferma_clean.select_dtypes(include=['number']).drop(columns=['Milk'])  
  
# vif_data = pd.DataFrame()  
# vif_data["feature"] = numeric_data.columns  
# vif_data["VIF"] = [variance_inflation_factor(numeric_data.values, i)  
#                   for i in range(numeric_data.shape[1])]  
  
# print("VIF анализ:")  
# display(vif_data.sort_values("VIF", ascending=False))
```

Мультиколлинеарность устранена.

Комментарий 🙌

Выводы с учётом того, что уже выполнено тобой в проекте, выше (то что вижу я ... для сопоставления наших позиций):

- можем сказать, что выводы те же, что и в пункте выше: есть не линейность с спо и жирностью во всех группах. Однозначно стоит создавать новые факторы.
- по мультиколлинеарности: скорее всего её нет (нет "узкого облака", похожего на четкую линейную зависимость).
- по эке - возможно иная зависимость между целевой и эке. Наверное надо попробовать "ввести нелинейность" (квадрат этого фактора), чтобы учесть кривизну. Результат (есть ли положительный эффект) проверим ниже, во 2ой линейной модели.
- отлично видно, что возраст влияет на удои. Более молодые коровы меньше дают молока.
- видно, что спо только для возрастных коров используется.

p.s. некоторые зависимости, которые я здесь описываю - я взял не просто из головы, а потому что сам этот проект выполнял (и scatterplot() "рисовал"). Если построишь аналогичные зависимости - увидишь подтверждение моим словам.

Комментарий 👉

Несколько моментов "на будущее" (сейчас забегаю чуть вперед, но будущем это будет важно) которые я хотел озвучить перед переходом к собственно исследованию (некоторые из этих моментов в твоём проекте реализованы):

- имеет смысл критично посмотреть на данные с точки зрения наличия пропусков в признаках (вопрос: все ли ml-модели "любят" пропуски?) и обработать или удалить их
- имеет смысл явно обозначить какую задачу мы будем решать: с учителем / без учителя, классификации / регрессии ...
- в моделях бинарной классификации (это как раз наш случай - предсказание вкуса молока во 2ой задаче!) имеет смысл смотреть на баланс классов. Два взгляда (с разных сторон) на эту проблему тут: <https://alexanderdyakonov.wordpress.com/2021/05/27/imbalance/> (<https://alexanderdyakonov.wordpress.com/2021/05/27/imbalance/>) и <https://habr.com/ru/post/349078/> (<https://habr.com/ru/post/349078/>)
- имеет смысл смотреть на корреляцию признаков. При высокой их степени коррелированности возникает проблема мультиколлинеарность признаков. Проблема "мультиколлинеарности" это и про переобучение модели и про её интерпретируемость и про сложность модели (содержание в модели избыточного количества переменных). В чем негативный эффект сложной модели? Вот в чем: усложняется интерпретация параметров, оценки параметров ненадежны – получаются большие стандартные ошибки, которые меняются с изменением объема наблюдений, что делает модель регрессии непригодной для прогнозирования.

► [Подробнее ...](#)

5. Обучение модели линейной регрессии

```
Ввод [33]: model_one = LinearRegression()

X_one = ferma_clean.drop('Milk', axis = 1)
y_one = ferma_clean['Milk']

X_train_one, X_test_one, y_train_one, y_test_one = train_test_split(
    X_one, y_one,
    test_size = 0.25,
    random_state = 42
)

num_cols_one = [col for col in X_one.select_dtypes('number').columns if col != 'Milk']
cat_cols_one = X_one.select_dtypes('object').columns.tolist()

encoder_one = OneHotEncoder(handle_unknown='ignore')
encoder_one.fit(X_train_one[cat_cols_one])
X_train_one_ohe = encoder_one.transform(X_train_one[cat_cols_one])
X_test_one_ohe = encoder_one.transform(X_test_one[cat_cols_one])

scaler_one = StandardScaler()
scaler_one.fit(X_train_one[num_cols_one])
X_train_one_sca = scaler_one.transform(X_train_one[num_cols_one])
X_test_one_sca = scaler_one.transform(X_test_one[num_cols_one])

X_train_one = hstack([X_train_one_ohe, X_train_one_sca])
X_test_one = hstack([X_test_one_ohe, X_test_one_sca])

model_one.fit(X_train_one, y_train_one)

predictions_one = model_one.predict(X_test_one)
print('R2 первой модели:', r2_score(y_test_one, predictions_one).round(3))
```

R2 первой модели: 0.832

ОТЛИЧНО! 👍

1. Здесь хорошо, происходит фиксация `random_state`. Воспроизводимость результатов разбиения выборки на обучающую (тренировочную) и тестовую мы обеспечили, значит при всех последующих запусках нашего кода подвыборки будут идентичными.

Комментарий 🙌

Я бы рекомендовал после разбиения данных на выборки посмотреть на их **размеры и размерности**. Метод `shape` для этого - идеальный помощник. "Цифры" по выборкам покажут верно ли мы произвели "разделение" данных.

p.s. так (**псевдокод**): `X_train.shape[0]` – «покажет» количество строк в тренировочной выборке, а `X_train.shape[1]` - количество столбцов в ней же. Ну а `X_train.shape` – выведет размерность `train`'а в виде кортежа с 2мя значениями (первое число – количество строк, второе – столбцов).

Необходимо исправить ❌

```
Ввод [27]: model_one = LinearRegression()

X_one = ferma_clean.drop('Milk', axis = 1)
y_one = ferma_clean['Milk']

X_train_one, X_test_one, y_train_one, y_test_one = train_test_split(
    X_one, y_one,
    test_size = 0.25,
    random_state = 42
)

num_cols_one = [col for col in X_one.select_dtypes('number').columns]
cat_cols_one = X_one.select_dtypes('object').columns.tolist()

encoder_one = OneHotEncoder()
encoder_one.fit(X_train_one[cat_cols_one])
X_train_one_ohe = encoder_one.transform(X_train_one[cat_cols_one])
X_test_one_ohe = encoder_one.transform(X_test_one[cat_cols_one])
```

Стоит настроить работу энкодера `OneHotEncoder` на работу с неизвестными значениями признаков - сейчас энкодер "при встрече с неизвестной категорией" (это может случиться в проде) - будет выбрасывать ошибку. Для непрерывности процесса моделирования **следует обязательно** настроить параметр `handle_unknown='ignore'`, чтобы новые, ранее неизвестные (на этапе обучения модели) значения признаков игнорировались и не включались в итоговую матрицу. Когда ты "включишь" параметр `handle_unknown='ignore'`, то параметра `drop='first'` рядом с ним быть НЕ ДОЛЖНО.

Почему параметра `drop='first'` НЕ ДОЛЖНО быть рядом с `handle_unknown='ignore'`? Во-первых, эти 2 параметра не работают вместе на той версии `scikit-learn` (0.24), которая развернута сейчас на Платформе. Во-вторых, при их одновременном использовании будет наблюдаться следующая проблема: и пропущенная (`dropped`) первая категория, и любые неизвестные категории будут кодироваться одинаково - вектором из всех нулей. Соответственно станет невозможным различить их после трансформации признаков.

ВАЖНО: параметр `drop='first'` может работать только (если параметр `handle_unknown='ignore'`) на последней версии пакета `scikit-learn`.

Все энкодеры донастроены.

КОММЕНТАРИЙ V2

Исправлено.

ОТЛИЧНО! 👍

Здесь всё верно. Масштабирование "обучается" на тренировочной выборке, а затем РЕЗУЛЬТАТ (обычно это `scaler`) последовательно применяется к тренировочной и к тестовой выборкам (и к валидационной тоже, если она имеется).

Для чего нужно масштабировать признаки перед подачей в линейную модель машинного обучения (например в линейную регрессию)? Нам нужно минимизировать функцию потерь методом градиентного спуска. В случае разных масштабов признаков (например, год рождения и количество детей) её линии уровня будут иметь вид вытянутых эллипсов. Тогда вектор антиградиента и направление от текущей точки к минимуму функции потерь могут не совпадать, мы можем уйти далеко и не в ту сторону, и, в зависимости от шага градиентного спуска, либо придётся сделать больше итераций, либо вообще не будет сходимости. Если же признаки отмасштабированы, то линии уровня похожи на окружности. И проблема несовпадения антиградиента и направления к минимуму не так выражена.

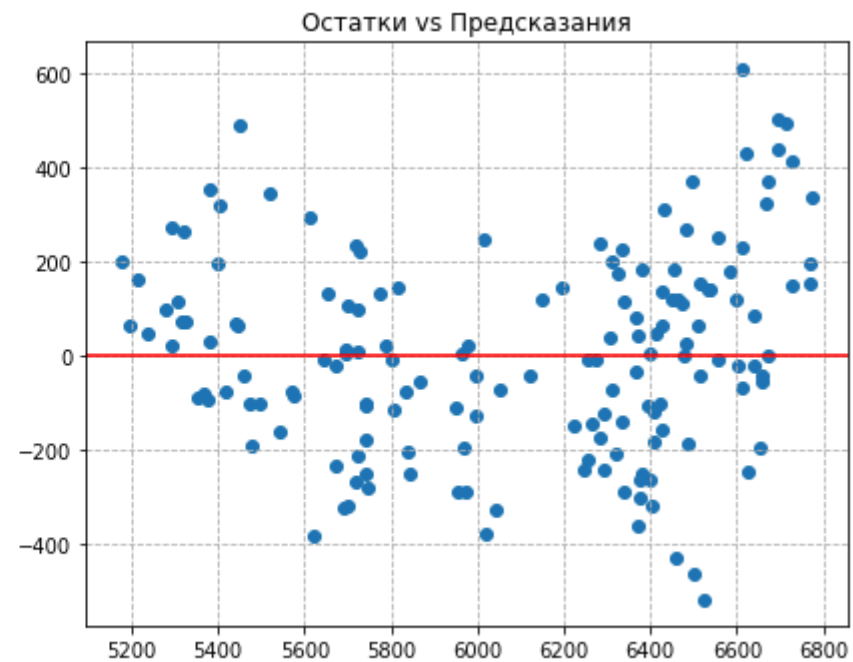
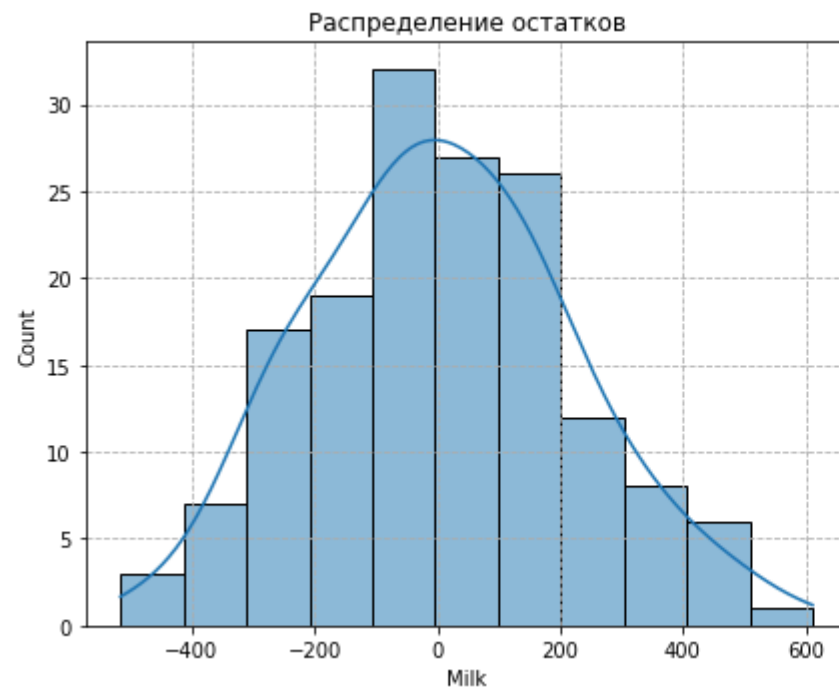
p.s. Для "деревянных моделей" масштабирование данных не обязательно, так как они не зависят от расстояния между объектами или величины признаков (суть работы "деревянных моделей" - разбиение пространства признаков на области, в которых принимается одно решение).

Посмотрим остатки первой модели.

```
Ввод [34]: residuals_one = y_test_one - predictions_one
```

```
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
sns.histplot(residuals_one, kde=True)
plt.title('Распределение остатков')
plt.grid(True, linestyle = '--')

plt.subplot(1, 2, 2)
plt.scatter(predictions_one, residuals_one)
plt.axhline(0, color='red')
plt.title('Остатки vs Предсказания')
plt.tight_layout()
plt.grid(True, linestyle = '--')
```



Распределение относительно равномерное.

Комментарий 🙌

Выводы (то что вижу я ... для сопоставления наших позиций):

- R2 высокий. Это хорошо, но можно попытаться "вырастить" R2 ещё больше

- Анализ остатков модели: есть участки, где ошибки неустойчивы. До 6000 - один уровень разброса. Более 6000 - разброс ошибки выше. Однозначно модель использовать нельзя - наблюдается гетероскедастичность (не постоянство дисперсии)
- Гистограмма - "колокол". Но есть локальные всплески. Которые тоже говорят о том, что модель можно улучшать

Общий вывод: модель можно и нужно улучшать. В EDA мы видели, что факторы 'спо' и 'эке' имеют не линейную связь с целевой. Поэтому их стоит преобразовать ('эке' $** 2$). Из 'спо' же сделаем категориальную переменную. Для этого надо ещё раз внимательно посмотреть на границы возможных категорий.

Строим вторую модель.

```
Ввод [35]: # сделаем бинарное значение в SP0 и возведем в квадрат Energy
ferma_clean_two = ferma_clean
ferma_clean_two['SP0_2'] = (ferma_clean_two['SP0'] > 0.9).astype(int)
ferma_clean_two = ferma_clean_two.drop(columns = 'SP0')
ferma_clean_two['Energy'] = ferma_clean_two['Energy'] **2
```

```
Ввод [36]: model_two = LinearRegression()

X_two = ferma_clean_two.drop('Milk', axis = 1)
y_two = ferma_clean_two['Milk']

X_train_two, X_test_two, y_train_two, y_test_two = train_test_split(
    X_two, y_two,
    test_size = 0.25,
    random_state = 42
)

num_cols_two = [col for col in X_two.select_dtypes('number').columns if col != 'Milk']
cat_cols_two = X_two.select_dtypes('object').columns.tolist()

encoder_two = OneHotEncoder(handle_unknown='ignore')
encoder_two.fit(X_train_two[cat_cols_two])
X_train_two_ohe = encoder_two.transform(X_train_two[cat_cols_two])
X_test_two_ohe = encoder_two.transform(X_test_two[cat_cols_two])

scaler_two = StandardScaler()
scaler_two.fit(X_train_two[num_cols_two])
X_train_two_sca = scaler_two.transform(X_train_two[num_cols_two])
X_test_two_sca = scaler_two.transform(X_test_two[num_cols_two])

X_train_two = hstack([X_train_two_ohe, X_train_two_sca])
X_test_two = hstack([X_test_two_ohe, X_test_two_sca])

model_two.fit(X_train_two, y_train_two)

predictions_two = model_two.predict(X_test_two)
print('R2 второй модели:', r2_score(y_test_two, predictions_two).round(3))
```

R2 второй модели: 0.844

Комментарий 🙌

Отлично, 5 шагов (EDA, обработка аномалий и пропусков, разделение данных на выборки, масштабирование/нормировку и кодирование признаков), входящие в типичный пайплайн подготовки данных **к этому моменту должны быть выполнены** ...

Я тебе совет дам: создай шаблон кода (или **ЛУЧШЕ - функцию/функции**), которые будут работать с момента загрузки данных и до получения выборов, готовых к передаче в модели МО.

Зачем такой такая функция/функции нужна? Чтобы не ломать всякий раз голову, не вспоминать и писать руками один и тот же код (да ещё и с возможностью совершить "глупую" ошибку). В функции(ях) должны быть (как минимум) следующие рутинные процедуры, которые встречаются ну абсолютно в каждом МО-проекте:

1. разбиение данных на выборки,
2. масштабирование/нормировка.
3. обработка категориальных признаков,

Затем ты эту функцию(и) будешь вставлять в каждый новый проект, чуть модифицировать его (имена переменных, названия столбцов) и гарантированно получать работающий и безошибочный код.

Комментарий 🙌

Присмотрись к использованию трансформеров и пайплайнов в машинном обучении. Да, это расширенный и поначалу сложный и непонятный концепт, НО ... вот их преимущества при использовании в коде:

Преимущества использования трансформеров:

1. **Гибкость и модульность:** Трансформеры позволяют разбить сложные операции по предобработке данных на более мелкие и легко управляемые компоненты. Это делает код более читаемым и модульным.
2. **Переиспользование:** Трансформеры могут быть повторно использованы на разных этапах анализа данных и построения моделей. Например, ты можешь создать трансформер для нормализации данных и использовать его как в обучении модели, так и на этапе инференса.
3. **Работа с разными типами данных:** Трансформеры могут обрабатывать различные типы данных, включая текст, изображения и числовые значения. Это позволяет тебе создавать единые интерфейсы для предобработки разнообразных данных.

Преимущества использования пайплайнов:

1. **Структурированность:** Пайплайны помогают организовать последовательность шагов обработки данных, начиная с загрузки и заканчивая подготовкой к обучению модели. Это обеспечивает структурированность и позволяет легко вносить изменения.
2. **Поддержание кода:** Пайплайны сокращают вероятность ошибок и упрощают поддержание кода. Отслеживание последовательности операций делает процесс отладки и оптимизации более прозрачным.
3. **Масштабируемость:** Пайплайны могут быть легко расширены новыми этапами обработки данных. Это позволяет добавлять новые операции без переписывания всего кода.

Объединение трансформеров и пайплайнов позволяет создавать чистые, структурированные и гибкие рабочие процессы для подготовки данных. Например, ты можешь создать трансформеры для преобразования текстовых данных и числовых значений, а затем построить пайплайн, который последовательно применяет эти трансформеры и завершается обучением модели. Это существенно упростит управление и анализ данных перед обучением моделей машинного обучения.

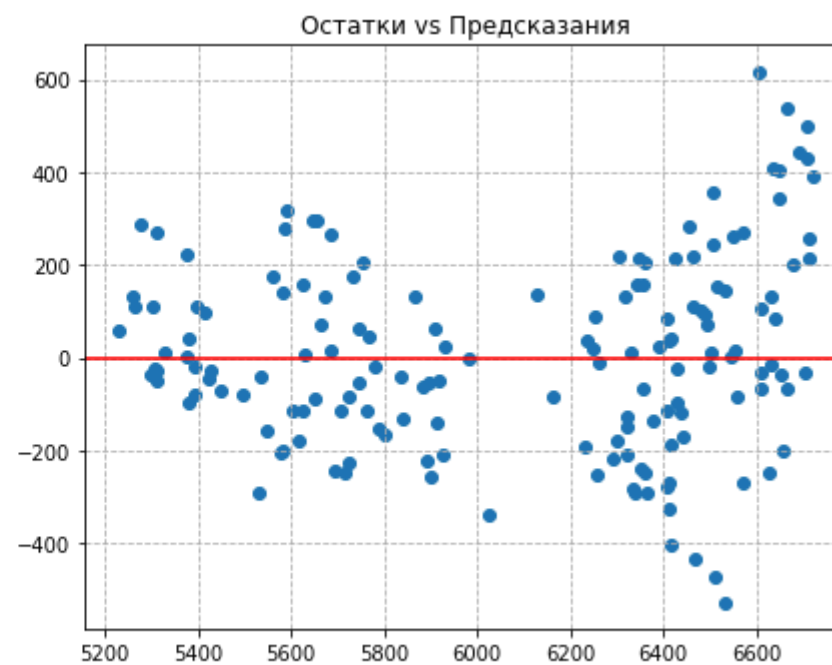
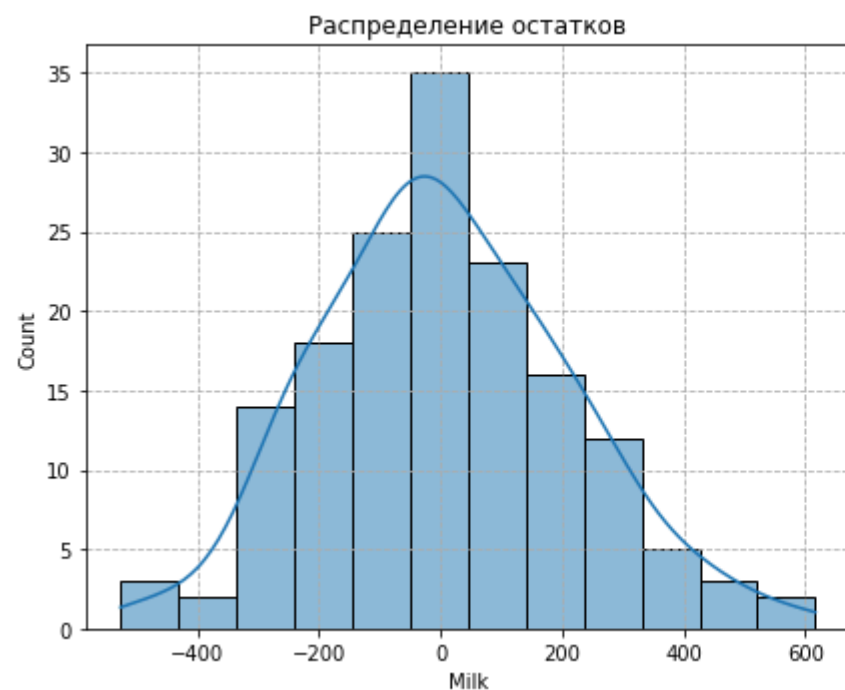
Несколько ссылок (нужен *url* для подключения к сайтам + возможно понадобится поиском поискать названия статей - если они были перемещены), которые позволят погрузиться в тему:

- [трансформеры](https://scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html) (<https://scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html>) и [пайплайны](https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html) (<https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html>), для организации предобработки (кодирования, масштабирования) и модели
- применение пайплайна [A Basic Introduction to Pipelines in Scikit Learn](https://towardsdatascience.com/a-basic-introduction-to-pipelines-in-scikit-learn-bd4cee34ad95) (<https://towardsdatascience.com/a-basic-introduction-to-pipelines-in-scikit-learn-bd4cee34ad95>).
- большой гайд как можно комбинировать разные этапы предобработки [Pipelines and composite estimators](https://scikit-learn.org/stable/modules/compose.html) (<https://scikit-learn.org/stable/modules/compose.html>).
- или можно написать свой класс-трансформер [Custom Transformers and Pipelines in Python](https://towardsdatascience.com/custom-transformers-in-python-part-ii-6fe11fc82e4) (<https://towardsdatascience.com/custom-transformers-in-python-part-ii-6fe11fc82e4>).

Ввод [37]: `residuals_two = y_test_two - predictions_two`

```
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
sns.histplot(residuals_two, kde=True)
plt.title('Распределение остатков')
plt.grid(True, linestyle = '--')

plt.subplot(1, 2, 2)
plt.scatter(predictions_two, residuals_two)
plt.axhline(0, color='red')
plt.title('Остатки vs Предсказания')
plt.tight_layout()
plt.grid(True, linestyle = '--')
```



R2 вырос. Распределение аналогичное.

Комментарий 🙌

Выводы (то что вижу я ... для сопоставления наших позиций):

- R2 увеличился на несколько пунктов. Здорово.

- Анализ остатков модели: дисперсия разброса ошибки стала чуть стабильнее на всём диапазоне фактических данных. Это явное улучшение. И это важно. Наблюдаются две зоны ошибок. Но разница дисперсии ошибок не систематическая, а выборочная.
- Гистограмма: среднее и медиана совпадают, но распределение не нормальное (бимодальности, плюс смещение среднего влево от нуля). Значит потенциал для улучшения модели всё ещё есть.

Общий вывод: Модель уже пригодна для прогнозирования.

Создаем третью модель.

```
Ввод [38]: # добавим таблицу ferma_dad
ferma_clean_three = ferma_clean_two
ferma_clean_three['id'] = ferma_main['id']
ferma_clean_three = ferma_clean_three.merge(ferma_dad, on = 'id', how = 'outer')
ferma_clean_three = ferma_clean_three.drop(columns = ['id'])
```

```
Ввод [39]: model_three = LinearRegression()

X_three = ferma_clean_three.drop('Milk', axis = 1)
y_three = ferma_clean_three['Milk']

X_train_three, X_test_three, y_train_three, y_test_three = train_test_split(
    X_three, y_three,
    test_size = 0.25,
    random_state = 42
)

num_cols_three = [col for col in X_three.select_dtypes('number').columns if col != 'Milk']
cat_cols_three = X_three.select_dtypes('object').columns.tolist()

encoder_three = OneHotEncoder(handle_unknown='ignore')
encoder_three.fit(X_train_three[cat_cols_three])
X_train_three_ohe = encoder_three.transform(X_train_three[cat_cols_three])
X_test_three_ohe = encoder_three.transform(X_test_three[cat_cols_three])

scaler_three = StandardScaler()
scaler_three.fit(X_train_three[num_cols_three])
X_train_three_sca = scaler_three.transform(X_train_three[num_cols_three])
X_test_three_sca = scaler_three.transform(X_test_three[num_cols_three])

X_train_three = hstack([X_train_three_ohe, X_train_three_sca])
X_test_three = hstack([X_test_three_ohe, X_test_three_sca])

model_three.fit(X_train_three, y_train_three)

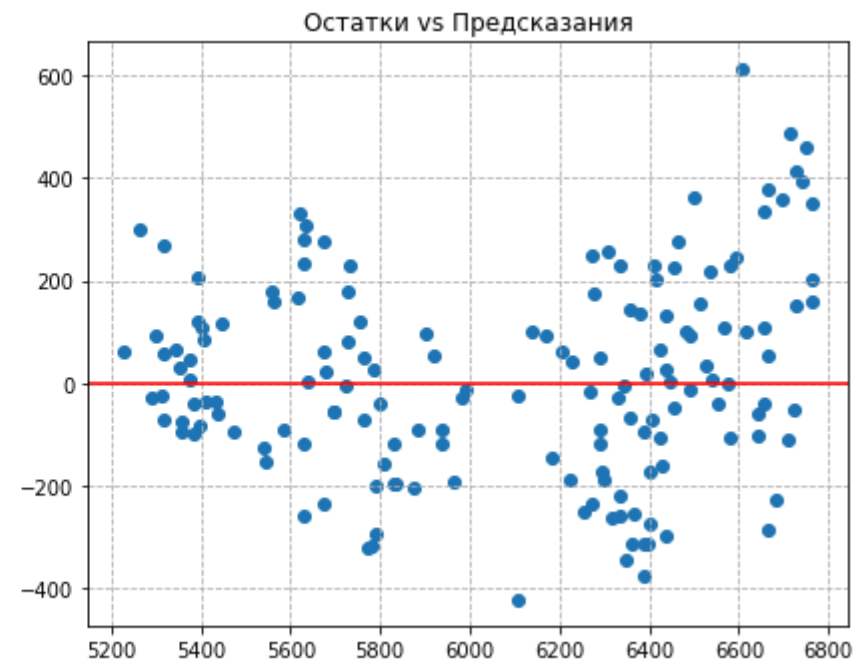
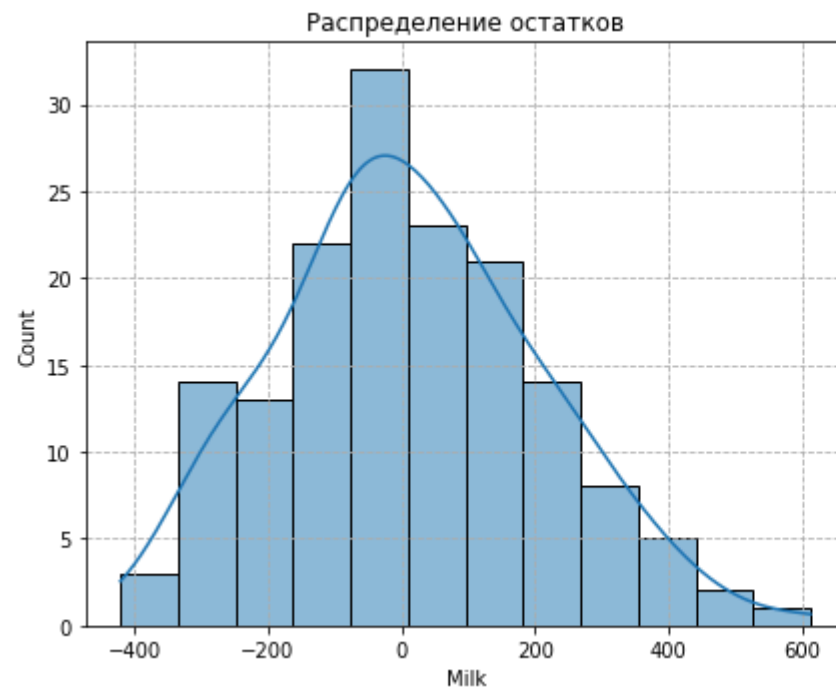
predictions_three = model_three.predict(X_test_three)
print('R2 третьей модели:', r2_score(y_test_three, predictions_three).round(3))
```

R2 третьей модели: 0.861

Ввод [40]: `residuals_three = y_test_three - predictions_three`

```
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
sns.histplot(residuals_three, kde=True)
plt.title('Распределение остатков')
plt.grid(True, linestyle = '--')

plt.subplot(1, 2, 2)
plt.scatter(predictions_three, residuals_three)
plt.axhline(0, color='red')
plt.title('Остатки vs Предсказания')
plt.tight_layout()
plt.grid(True, linestyle = '--')
```



R2 вырос. Остатки аналогичны.

Комментарий 🙋

Выводы (то что вижу я ... для сопоставления наших позиций):

- R2 ещё увеличился! Наши факторы на 85,1% объясняют разброс в целевой переменной. Это успех.

- Анализ остатков модели: дисперсия разброса ошибки осталась стабильной. Есть несколько "выдающихся" точек на высоких удоях, но это не системные дела, а разовая акция похоже. Можем признать, что ошибка равномерная на всём диапазоне значений. Т.е. остатки - гомоскедастичны. Это важно. Значит модель МОЖНО использовать для прогноза.
- Гистограмма: среднее и медиана совпадают, не смещены относительно 0. Но бимодальность присутствуют.

Общий вывод: Потенциал для улучшения модели всё ещё есть, но на текущий момент признаём модель качественной (остатки гомоскедастичны) и пригодной для работы. Посмотрим на точность модели.

```

Ввод [41]: def calculate_metrics(y_true, y_pred, model_name):
    return {
        'Модель': model_name,
        'R²': r2_score(y_true, y_pred),
        'MAE': mean_absolute_error(y_true, y_pred),
        'RMSE': mean_squared_error(y_true, y_pred, squared=False)
    }

metrics = [
    calculate_metrics(y_test_one, predictions_one, "Модель 1 (базовая)"),
    calculate_metrics(y_test_two, predictions_two, "Модель 2 (SPO_2 + Energy²)"),
    calculate_metrics(y_test_three, predictions_three, "Модель 3 (с данными о папе)")
]

results = pd.DataFrame(metrics).set_index('Модель')

def highlight_max(s):
    is_max = s == s.max()
    return ['background-color: #98FB98' if v else '' for v in is_max]

styled_results = results.style \
    .format({'R²': '{:.3f}', 'MAE': '{:.1f}', 'RMSE': '{:.1f}'}) \
    .apply(highlight_max, subset=['R²']) \
    .set_caption('Сравнение моделей линейной регрессии') \
    .set_properties(**{'text-align': 'center'})

display(styled_results)

```

Сравнение моделей линейной регрессии

	R²	MAE	RMSE
Модель			
Модель 1 (базовая)	0.832	172.1	214.7
Модель 2 (SPO_2 + Energy²)	0.844	163.6	206.7
Модель 3 (с данными о папе)	0.861	155.7	195.0

Лучшая модель 3.

```
Ввод [42]: std_error = residuals_three.std()
confidence = 1.96 * std_error

print(f"Средний доверительный интервал: ±{confidence:.1f} кг")
```

Средний доверительный интервал: ±382.8 кг

ОТЛИЧНО! 👍

Есть, вижу расчет доверительных интервалов.

Далее с помощью 3 модели, с прогнозируем удой коров, которых фермер хочет купить.

```
Ввод [43]: cow_buy_model_one = cow_buy
cow_buy_model_one['Energy'] = ferma_clean_three['Energy'].mean() * 1.05
cow_buy_model_one['SP0_2'] = ((ferma_main['SP0'].mean() * 1.05) > 0.9).astype(int)

cow_buy['Fat'] = cow_buy['Fat'].apply(categorize_fat)
```

ОТЛИЧНО! 👍

Отлично, устанавливаешь параметры кормления для наших потенциально новых коровок (в соответствии с требованиями фермера).

```
Ввод [44]: num_cols_cow = ['Energy', 'Prot in milk', 'Age', 'SP0_2']
cat_cols_cow = ['Breed', 'Pasture', 'Father breed', 'Fat', 'Father name']

cow_buy_cat = encoder_three.transform(cow_buy_model_one[cat_cols_cow])
cow_buy_num = scaler_three.transform(cow_buy_model_one[num_cols_cow])
cow_buy_good = hstack([cow_buy_cat, cow_buy_num])

cow_buy_model_one['Milk'] = model_three.predict(cow_buy_good)
```

Ввод [45]: cow_buy_model_one

Out[45]:

	Breed	Pasture	Father breed	Father name	Fat	Prot in milk	Age	Energy	SPO_2	Milk
0	Вис Бик Айдиал	Холмистое	Айдиал	Геркулес	low	3.076	1	223.889831	1	6393.939331
1	Вис Бик Айдиал	Равнинное	Соверин	Буйный	low	3.081	0	223.889831	1	6098.893890
2	РефлешнСоверинг	Равнинное	Соверин	Барин	low	3.074	1	223.889831	1	6097.122467
3	РефлешнСоверинг	Холмистое	Айдиал	Буйный	low	3.061	1	223.889831	1	6504.705669
4	РефлешнСоверинг	Равнинное	Айдиал	Буйный	medium	3.074	1	223.889831	1	6390.938262
5	РефлешнСоверинг	Равнинное	Соверин	Геркулес	medium	3.053	0	223.889831	1	6242.518246
6	Вис Бик Айдиал	Холмистое	Айдиал	Геркулес	low	3.076	1	223.889831	1	6393.939331
7	Вис Бик Айдиал	Равнинное	Соверин	Буйный	low	3.079	1	223.889831	1	6322.105434
8	РефлешнСоверинг	Равнинное	Соверин	Барин	low	3.074	1	223.889831	1	6097.122467
9	РефлешнСоверинг	Холмистое	Айдиал	Буйный	low	3.079	0	223.889831	1	6213.120519
10	Вис Бик Айдиал	Холмистое	Соверин	Соловчик	high	3.055	1	223.889831	1	6449.245485
11	РефлешнСоверинг	Равнинное	Соверин	Геркулес	high	3.073	1	223.889831	1	6522.281460
12	Вис Бик Айдиал	Холмистое	Айдиал	Геркулес	medium	3.046	1	223.889831	1	6539.200655
13	Вис Бик Айдиал	Холмистое	Соверин	Барин	low	3.075	0	223.889831	1	5926.671033
14	РефлешнСоверинг	Равнинное	Соверин	Барин	low	3.074	1	223.889831	1	6097.122467
15	РефлешнСоверинг	Холмистое	Айдиал	Буйный	low	3.065	1	223.889831	1	6487.612267
16	Вис Бик Айдиал	Холмистое	Соверин	Соловчик	high	3.079	1	223.889831	1	6346.685077
17	РефлешнСоверинг	Равнинное	Соверин	Геркулес	high	3.053	0	223.889831	1	6393.083624
18	Вис Бик Айдиал	Равнинное	Соверин	Буйный	low	3.069	1	223.889831	1	6364.838937
19	РефлешнСоверинг	Равнинное	Айдиал	Буйный	medium	3.074	1	223.889831	1	6390.938262

Прогноз удоя для новых коров находится в диапазоне 5926 – 6536 кг/год, что соответствует данным фермы (5 200–7 200 кг).

Лучшие кандидаты (удой >6 383 кг):

Порода: «Вис Бик Айдиал»

Пастбище: Холмистое

Возраст: старше 2 лет

Отец: «Айдиал» (дает +150–200 кг к удою).

ОТЛИЧНО! 👍

Да, расчет метрик, доверительных интервалов - всё по букве брифа. Ну и выбор в пользу Зей модели аргументирован и посему очевиден. Предсказание что находятся коровы, которые дают > 6.000 кг молока - верно (в эталонном проекте автора этого спринта всё точно также). Итак, с линейными моделями всё, приступаем к логистической регрессии, будем предсказывать вкус молока.

6. Обучение модели логистической регрессии

Приступаем к задаче классификации. Основная задача сделать предсказание, будет ли молоко вкусным.

Подготовим датафрейм и посмотрим еще раз корреляцию связанную со вкусом молока.

Ввод [46]:

```
# добавим столбец порода папы быка и удалим столбцы с большой мультиколлинеарностью и с нулевой корреляцией
ferma_clean_class = ferma_main
ferma_clean_class = ferma_clean_class.merge(ferma_dad, on = 'id', how = 'outer')
ferma_clean_class = ferma_clean_class.drop(columns = [
    'id', 'Milk', 'Father breed',
    'Father name', 'Breed', 'Prot in milk',
    'Protein'])
```

Комментарий 🙋

В случае объединения/слияния датафреймов рекомендую тебе смотреть на размерности датафреймов ДО слияния и ПОСЛЕ него. В подобных операциях часто ошибки бывают.

Ввод [47]:

ferma_clean_class

Out[47]:

	Energy	SPO	Pasture	Fat	Flavor	Age
0	14.2	0.890	Равнинное	low	1	1
1	12.8	0.890	Равнинное	low	1	0
2	14.0	0.885	Холмистое	low	0	1
3	12.4	0.885	Холмистое	low	0	1
4	12.8	0.885	Равнинное	high	1	0
...	...	...	...	...	...	...
624	15.4	0.940	Равнинное	high	0	1
625	12.9	0.880	Равнинное	medium	1	0
626	14.4	0.880	Холмистое	medium	0	1
627	13.8	0.890	Равнинное	medium	1	1
628	16.2	0.940	Холмистое	high	1	1

629 rows × 6 columns

Ввод [48]:

interval_cols_cow = ['Energy', 'SPO']

ferma_clean_class.phik_matrix(interval_cols = interval_cols_cow)

Out[48]:

	Energy	SPO	Pasture	Fat	Flavor	Age
Energy	1.000000	0.589307	0.103065	0.682525	0.271287	0.768438
SPO	0.589307	1.000000	0.158532	0.660629	0.569538	0.744653
Pasture	0.103065	0.158532	1.000000	0.000000	0.270583	0.129482
Fat	0.682525	0.660629	0.000000	1.000000	0.085499	0.284897
Flavor	0.271287	0.569538	0.270583	0.085499	1.000000	0.166869
Age	0.768438	0.744653	0.129482	0.284897	0.166869	1.000000

Необходимо исправить ❌

При использовании **phik** крайне важно настроить параметр **interval_cols**, передав ему список с названиями непрерывных признаков: если этого не сделать, библиотека своими силами будет пытаться разнести признаки по типам, и она часто делает в этом ошибки - от этого в свою

КОММЕНТАРИЙ V2

Исправлено.

```
Ввод [49]: # from statsmodels.stats.outliers_influence import variance_inflation_factor

# numeric_data_new = ferma_clean_class.select_dtypes(include=['number'])

# vif_data = pd.DataFrame()
# vif_data["feature"] = numeric_data_new.columns
# vif_data["VIF"] = [variance_inflation_factor(numeric_data_new.values, i)
#                   for i in range(numeric_data_new.shape[1])]

# print("VIF анализ:")
# display(vif_data.sort_values("VIF", ascending=False))
```

Приступаем к созданию модели.

```
Ввод [50]: X = ferma_clean_class.drop(columns = ['Flavor'])
y = ferma_clean_class['Flavor']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size = 0.25,
    random_state = 42
)

encoder_class = OneHotEncoder(handle_unknown='ignore')
scaler_class = StandardScaler()

num_cols_class = ['SP0', 'Energy', 'Age']
cat_cols_class = ['Pasture', 'Fat']

encoder_class.fit(X_train[cat_cols_class])
X_train_ohe = encoder_class.transform(X_train[cat_cols_class])
X_test_ohe = encoder_class.transform(X_test[cat_cols_class])

scaler_class.fit(X_train[num_cols_class])
X_train_sca = scaler_class.transform(X_train[num_cols_class])
X_test_sca = scaler_class.transform(X_test[num_cols_class])

X_train_class = hstack([X_train_ohe, X_train_sca])
X_test_class = hstack([X_test_ohe, X_test_sca])

model_class = LogisticRegression()
model_class.fit(X_train_class, y_train)

y_pred = model_class.predict(X_test_class)
y_proba = model_class.predict_proba(X_test_class)
```

Необходимо исправить ❌

Стоит настроить работу энкодера OneHotEncoder на работу с неизвестными значениями признаков - сейчас энкодер "при встрече с неизвестной категорией" (это может случиться в проде) - будет выбрасывать ошибку.

КОММЕНТАРИЙ V2

Исправлено.

```
Ввод [51]: print('Accuracy:', accuracy_score(y_test, y_pred).round(2))
print('Precision:', precision_score(y_test, y_pred).round(2))
print('Recall:', recall_score(y_test, y_pred).round(2))
```

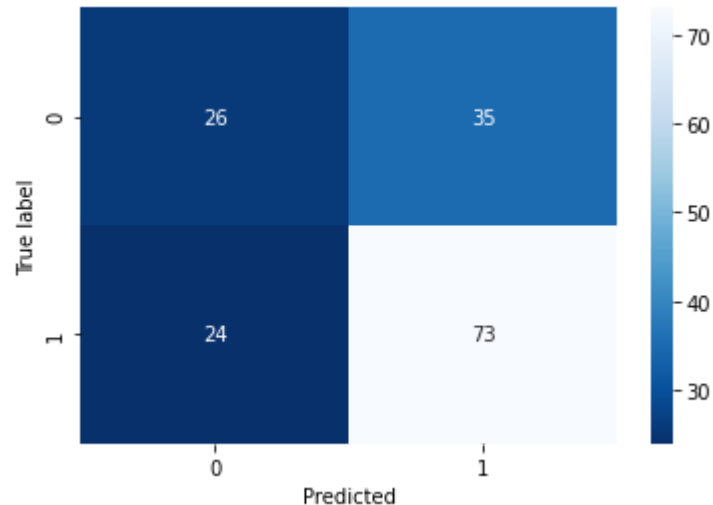
Accuracy: 0.63
Precision: 0.68
Recall: 0.75

ОТЛИЧНО! 👍

Ну и здесь ты молодец, что считаешь что-то из набора (полнота / точность / accuracy).

Матрица ошибок (confusion matrix) - тоже ОК, так как нам сейчас нужно будет определиться с бизнес-метрикой (как же фермеру стоит отбирать новых коров).

```
Ввод [52]: cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues_r')
plt.ylabel('True label')
plt.xlabel('Predicted');
```



По матрице ошибок: ошибка 1 рода, для фермера хуже, так как если молоко окажется невкусным, пострадает репутация.
Accuracy: 0.63 или 63% доля правильных ответов, не плохо, но хотелось бы больше.
Precision: 0.69 или 69% точность предсказаний, что молоко будем вкусным, относительно невкусного.
Recall: 0.74 или 74% точность предсказаний 1 класса, из всех объектов этого класса.

Попробуем свести ошибку 1 рода к нулю, для этого изменим значение порога. Цель увеличить Recall до максимума.

Необходимо исправить ❌

Попробуем свести ошибку 1 рода к нулю, для этого изменим значение порога. Цель увеличить Recall до максимума - максимизация recall уменьшает до 0 ошибку 2го рода, а не 1го.

КОММЕНТАРИЙ V2

Попробуем свести ошибку 1 рода к нулю, для этого изменим значение порога. Цель увеличить Recall до максимума - Артемий, эта фраза должна так звучать: *Попробуем свести ошибку 1 рода к нулю, для этого изменим значение порога. Цель увеличить Precision до максимума*

Необходимо исправить ❌

Давай подумаем на эту тему - а ту ли метрику ты используешь для решения задачи нашего фермера? Мне кажется, что повышать (иногда говорят "выращивать") нужно precision, уменьшая при этом FP. И вот когда FP станет == 0, то тогда precision станет == 1. И именно в этот момент наш фермер сможет доверять модели, которая со 100% гарантией будет отличать вкусное молоко от невкусного.

Мои дополнительные аргументы - на скрине:



(ложно-положительные ответы): Мы хотим купить из всего предлагаемого нам стада только тех коров, у которых вкусное молоко. Т.е. для нас важно, чтобы ошибка FP была минимальная у модели. Сейчас он на уровне 23%. Т.е. из 100 купленных коров нам могут попасться, всё же, 23, у которых молоко будет не вкусное. Это будет для нас убыток.

Комментарий 🙋

После исправления красных комментариев и цифры и результаты и финальный вывод ниже **полностью поменяются**. Необходимо поправить все ошибки выше, после чего, на 2ой итерации я продолжу проверку проекта с этого момента.

КОММЕНТАРИЙ V2

Вижу исправления ниже

```

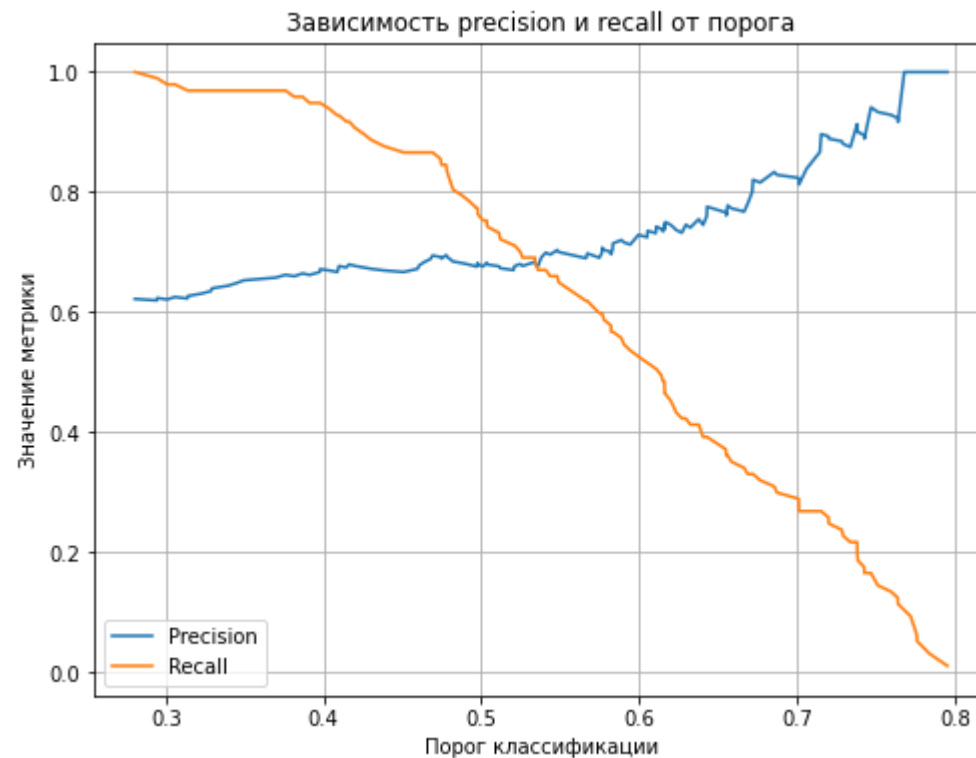
Ввод [53]: y_proba_positive = y_proba[:, 1] if len(y_proba.shape) == 2 else y_proba

precision, recall, thresholds = precision_recall_curve(y_test, y_proba_positive)

plt.figure(figsize=(8, 6))
plt.plot(thresholds, precision[:-1], label="Precision")
plt.plot(thresholds, recall[:-1], label="Recall")
plt.xlabel("Порог классификации")
plt.ylabel("Значение метрики")
plt.title("Зависимость precision и recall от порога")
plt.legend()
plt.grid(True)
plt.show()

optimal_idx = np.argmax(precision)
optimal_threshold = thresholds[optimal_idx]
print(f"Оптимальный порог: {optimal_threshold:.2f}")
print(f"Максимальная precision: {precision[optimal_idx]:.2f}")
print(f"Recall при этом пороге: {recall[optimal_idx]:.2f}")

```



Оптимальный порог: 0.77
Максимальная precision: 1.00
Recall при этом пороге: 0.10

Необходимо исправить ❌

```
Ввод [46]: y_proba = model_class.predict_proba(X_test_class)[:,-1]

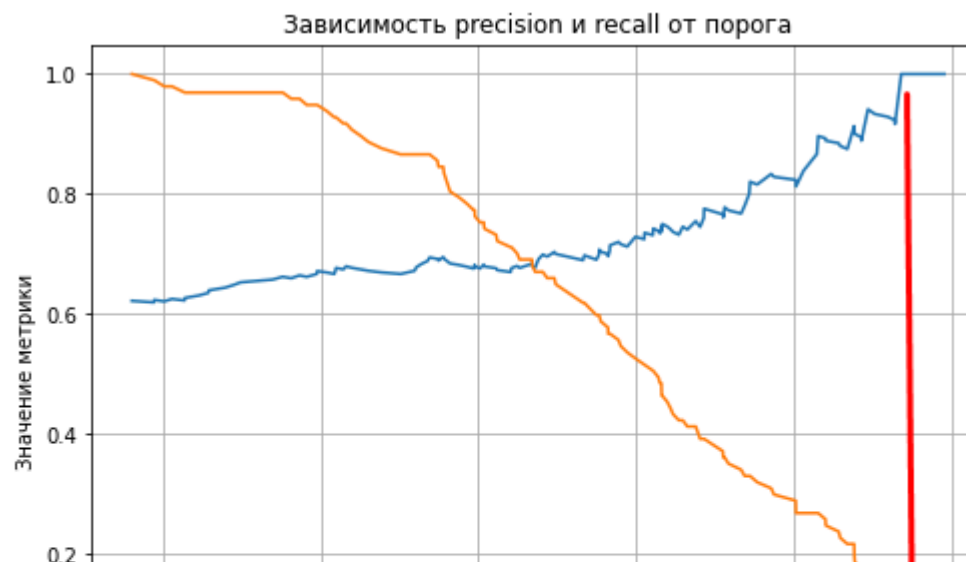
precision, recall, thresholds = precision_recall_curve(y_test, y_proba)
f1_scores = 2 * (precision * recall) / (precision + recall + 1e-9)
optimal_idx = np.argmax(f1_scores)
optimal_threshold = thresholds[optimal_idx]
print(f"Оптимальный порог: {optimal_threshold:.2f}")
```

Оптимальный порог: 0.33

Выше говоришь о максимизации recall, а в коде что максимизируешь?

КОММЕНТАРИЙ V2

ОТЛИЧНО! 👍



Применяем модель с новым порогом, на датафрейме коров, которых хочет купить фермер.

```
Ввод [54]: cow_buy_class = cow_buy
cow_buy_class['SP0'] = ferma_main['SP0'].mean() * 1.05
cow_buy_class['Energy'] = ferma_main['Energy'].mean() * 1.05

num_cols_class_cow = ['SP0', 'Fat', 'Energy', 'Age']
cat_cols_class_cow = ['Pasture']

X_cow_buy_cat = encoder_class.transform(cow_buy_class[cat_cols_class])
X_cow_buy_num = scaler_class.transform(cow_buy_class[num_cols_class])
X_cow_buy = hstack([X_cow_buy_cat, X_cow_buy_num])

y_proba_cow_buy = model_class.predict_proba(X_cow_buy)[: , 1]

optimal_threshold = 0.77

cow_buy_class['Flavor_pred'] = (y_proba_cow_buy >= optimal_threshold).astype(int)
```

Добавили в датафрейм столбец с предсказанием вкуса молока. А так же из предыдущего датафрейма добавим расчет удоя.

```
Ввод [55]: cow_buy_final = cow_buy
cow_buy_final['Flavor'] = cow_buy_class['Flavor_pred']
cow_buy_final['Milk'] = cow_buy_model_one['Milk']
cow_buy_final = cow_buy_final.drop(columns = ['Flavor_pred'])
cow_buy_final
```

Out[55]:

	Breed	Pasture	Father breed	Father name	Fat	Prot in milk	Age	Energy	SPO_2	Milk	SPO	Flavor
0	Вис Бик Айдиал	Холмистое	Айдиал	Геркулес	low	3.076	1	15.271073	1	6393.939331	0.958772	1
1	Вис Бик Айдиал	Равнинное	Соверин	Буйный	low	3.081	0	15.271073	1	6098.893890	0.958772	0
2	РефлешнСоверинг	Равнинное	Соверин	Барин	low	3.074	1	15.271073	1	6097.122467	0.958772	0
3	РефлешнСоверинг	Холмистое	Айдиал	Буйный	low	3.061	1	15.271073	1	6504.705669	0.958772	1
4	РефлешнСоверинг	Равнинное	Айдиал	Буйный	medium	3.074	1	15.271073	1	6390.938262	0.958772	0
5	РефлешнСоверинг	Равнинное	Соверин	Геркулес	medium	3.053	0	15.271073	1	6242.518246	0.958772	0
6	Вис Бик Айдиал	Холмистое	Айдиал	Геркулес	low	3.076	1	15.271073	1	6393.939331	0.958772	1
7	Вис Бик Айдиал	Равнинное	Соверин	Буйный	low	3.079	1	15.271073	1	6322.105434	0.958772	0
8	РефлешнСоверинг	Равнинное	Соверин	Барин	low	3.074	1	15.271073	1	6097.122467	0.958772	0
9	РефлешнСоверинг	Холмистое	Айдиал	Буйный	low	3.079	0	15.271073	1	6213.120519	0.958772	1
10	Вис Бик Айдиал	Холмистое	Соверин	Соловчик	high	3.055	1	15.271073	1	6449.245485	0.958772	0
11	РефлешнСоверинг	Равнинное	Соверин	Геркулес	high	3.073	1	15.271073	1	6522.281460	0.958772	0
12	Вис Бик Айдиал	Холмистое	Айдиал	Геркулес	medium	3.046	1	15.271073	1	6539.200655	0.958772	0
13	Вис Бик Айдиал	Холмистое	Соверин	Барин	low	3.075	0	15.271073	1	5926.671033	0.958772	1
14	РефлешнСоверинг	Равнинное	Соверин	Барин	low	3.074	1	15.271073	1	6097.122467	0.958772	0
15	РефлешнСоверинг	Холмистое	Айдиал	Буйный	low	3.065	1	15.271073	1	6487.612267	0.958772	1
16	Вис Бик Айдиал	Холмистое	Соверин	Соловчик	high	3.079	1	15.271073	1	6346.685077	0.958772	0
17	РефлешнСоверинг	Равнинное	Соверин	Геркулес	high	3.053	0	15.271073	1	6393.083624	0.958772	0
18	Вис Бик Айдиал	Равнинное	Соверин	Буйный	low	3.069	1	15.271073	1	6364.838937	0.958772	0
19	РефлешнСоверинг	Равнинное	Айдиал	Буйный	medium	3.074	1	15.271073	1	6390.938262	0.958772	0

7. Итоговые выводы

```
Ввод [56]: best_cows = cow_buy[(cow_buy_final['Milk'] > 6383) & (cow_buy_final['Flavor'] == 1)]  
display('Рекомендация купить:', best_cows)
```

'Рекомендация купить:'

	Breed	Pasture	Father breed	Father name	Fat	Prot in milk	Age	Energy	SPO_2	Milk	SPO	Flavor_pred	Flavor
0	Вис Бик Айдиал	Холмистое	Айдиал	Геркулес	low	3.076	1	15.271073	1	6393.939331	0.958772	1	1
3	РефлешнСоверинг	Холмистое	Айдиал	Буйный	low	3.061	1	15.271073	1	6504.705669	0.958772	1	1
6	Вис Бик Айдиал	Холмистое	Айдиал	Геркулес	low	3.076	1	15.271073	1	6393.939331	0.958772	1	1
15	РефлешнСоверинг	Холмистое	Айдиал	Буйный	low	3.065	1	15.271073	1	6487.612267	0.958772	1	1

Рекомендации по покупке коров:

1. **Подходящие коровы:** 4 из 20.

2. **Лучшие параметры:**

- Порода: Вис Бик Айдиал.
- Пастбище: Холмистое.
- Отец: Айдиал.
- Возраст: >2 лет.

3. **Ожидаемый удой:** 6000–6500 кг/год.

4. **Риски:**

- 1–2 коровы могут дать молоко «неидеального» вкуса (но это редко).

Действия:

- Купить отобранных 4 коровы.
- Увеличить ЭКЕ корма на 5%.
- Контролировать СПО (0.9).

Результат ревью:

Я отмечаю адекватный уровень твоей программистской подготовки. Ты используешь продвинутые концепции языка Python, библиотеки pandas и одной из основных библиотек машинного обучения scikit-learn. Всё это в целом упрощает твой код и одновременно показывает твой уровень владения инструментами анализа данных и machine learning.

С комментированием никаких проблем нет. Твоих объяснений достаточно для понимания твоими коллегами хода мыслей. Ссылки из брифа проекта в нужных местах также помогают следовать за развитием событий и дают полную информацию касаясь того, что будет происходить в том или ином разделе проекта.

Мне было просто проверять твой проект, потому что он чётко структурирован, с использованием оглавления.

Зелёным цветом я отметил и прокомментировал удачные и элегантные решения, на которые ты можешь опираться в будущих проектах.

Жёлтым цветом я выделил то, что в следующий раз можно сделать по-другому. Ты можешь учесть эти комментарии при выполнении будущих заданий или доработать проект сейчас.

Однако есть принципиальные моменты, которые не позволяют мне принять твой проект в его нынешнем состоянии:

Необходимо исправить ❌

1. (10) Датафрейм "с коровами для предсказаний" - это ведь по сути тестовая выборка, данные для предсказаний (для наших моделей). А в тестовой выборке мы не правомочны что либо удалять - эти данные поступают на вход обученной модели и по ним должны быть сделаны "предсказания".
2. (21) id из графического анализа нужно удалить.
3. (21) "Провалы" в значениях гистограммы реально существуют? Или они появляются ввиду ошибочно выбранного значения bins?
4. (22, 41) При использовании **phik** крайне важно настроить параметр **interval_cols**, передав ему список с названиями непрерывных признаков: если этого не сделать, библиотека своими силами будет пытаться разнести признаки по типам, и она часто делает в этом ошибки.
5. (27, 43) Стоит настроить работу энкодера OneHotEncoder на работу с неизвестными значениями признаков - сейчас энкодер "при встрече с неизвестной категорией" (это может случиться в проде) - будет выбрасывать ошибку. Для непрерывности процесса моделирования можно настроить параметр `handle_unknown='ignore'`, чтобы новые, ранее неизвестные значения игнорировались и не включались в итоговую матрицу.
6. (45) максимизация recall уменьшает до 0 ошибку 2го рода, а не 1го.
7. (45) Давай подумаем на эту тему - а ту ли метрику ты используешь для решения задачи нашего фермера?
8. (46) Выше говоришь о максимизации recall, а в коде что максимизируешь?

Эти моменты требуют твоего внимания. Комментарии по ним помечены красным цветом.

Подводя итог: хорошая работа на самом-то деле, молодец! Есть несколько моментов к исправлению, но уверен, что они не вызовут у тебя

